

Rozpoznawanie akcji na danych szkieletowych w oparciu o prostą ST-GCN

1. Cel projektu

Celem projektu było opracowanie i eksperymentalna walidacja metod rozpoznawania akcji na podstawie danych szkieletowych. Projekt realizowany był w dwóch etapach:

- **CV_Proj1:** Ekstrakcja danych szkieletowych przy użyciu MediaPipe oraz wstępne przetwarzanie danych na potrzeby klasyfikacji.
- **CV_Proj2:** Implementacja i testowanie modelu ST-GCN do klasyfikacji akcji na podstawie przetworzonych danych.

2. Skład grupy

- Tomasz Bochnak, Jakub Łubkowski, Marcin Miłucha, Szymon Słota, Paweł Steczkiewicz

3. Przebieg realizacji projektu

3.1. CV_Proj1: Ekstrakcja i wstępne przetwarzanie danych szkieletowych

W pierwszym etapie projektu zajęliśmy się przygotowaniem danych wejściowych dla modelu klasyfikacyjnego. W tym celu:

- Stworzono skrypt do wczytywania sekwencji obrazów z plików wideo oraz pojedynczych plików graficznych.
- Przetestowano różne modele MediaPipe do ekstrakcji danych szkieletowych i oceniono ich skuteczność.
- Zaimplementowano zapis wykrytych punktów szkieletowych do plików **JSON** oraz **CSV**.
- Stworzono metodę augmentacji danych szkieletowych, obejmującą losowe przesunięcia i skalowanie punktów kluczowych.
- Przygotowano skrypt do wizualizacji danych przed i po augmentacji w dwóch oknach.

Ze względu na nienajlepsze wyniki ekstrakcji punktów kluczowych z MediaPipe, porzuciliśmy tę metodę na rzecz innego podejścia w drugim etapie projektu.

3.2. CV_Proj2: Implementacja klasyfikacji za pomocą ST-GCN

Drugi etap projektu obejmował implementację modelu ST-GCN, który został przystosowany do analizy danych szkieletowych:

- W CV_Proj2 wykorzystano inną klasę z MediaPipe do generowania key pointów, co pozwoliło uzyskać lepsze rezultaty.
- Zamiast przetwarzania pojedynczych obrazów, zaimplementowano rozbijanie plików wideo na klatki, a następnie przypisywanie każdej klatce odpowiedniej klasy ręcznie.
- Przetworzono dane do postaci odpowiedniej dla modeli grafowych.

- Zaimplementowano model ST-GCN w oparciu o bibliotekę torch-geometric.
- Stworzono skrypt do uczenia modelu na przetworzonych danych oraz do testowania jego skuteczności.
- Przeprowadzono trening oraz testy modelu, uzyskując dokładność 77,41% na zbiorze testowym.

4. Podsumowanie

W ramach projektu opracowano pipeline przetwarzania danych szkieletowych oraz ich klasyfikacji za pomocą modelu ST-GCN. Zastosowanie alternatywnej biblioteki do generowania punktów kluczowych oraz podejście oparte na analizie klatek z wideo pozwoliły uzyskać całkiem niezłe wyniki.

ogwvwpuxuy

December 5, 2024

```
[ ]: from __future__ import absolute_import
from __future__ import division
from __future__ import print_function

__all__ = ['cv2_imshow', 'cv_imshow']

import cv2
from IPython import display
import PIL

def cv2_imshow(a):

    a = a.clip(0, 255).astype('uint8')
    if a.ndim == 3:
        if a.shape[2] == 4:
            a = cv2.cvtColor(a, cv2.COLOR_BGRA2RGBA)
        else:
            a = cv2.cvtColor(a, cv2.COLOR_BGR2RGB)
    display.display(PIL.Image.fromarray(a))

cv_imshow = cv2_imshow

def clear_folder(folder_path):
    """Usuń wszystkie pliki w danym folderze."""
    for filename in os.listdir(folder_path):
        file_path = os.path.join(folder_path, filename)
        try:
            if os.path.isfile(file_path):
                os.remove(file_path)
            elif os.path.isdir(file_path):
                os.rmdir(file_path)
        except Exception as e:
            print(f"Nie udało się usunąć {file_path}. Błąd: {e}")
```

```
[ ]: import cv2
import mediapipe as mp
import os
import json
import glob

# Inicjalizacja MediaPipe Pose
mp_pose = mp.solutions.pose
pose = mp_pose.Pose(min_detection_confidence=0.5, min_tracking_confidence=0.5)

def extract_keypoints_from_image(image):
    """Ekstrakcja punktów kluczowych (keypoints) z obrazu"""
    results = pose.process(cv2.cvtColor(image, cv2.COLOR_BGR2RGB))
    keypoints = []
    if results.pose_landmarks:
        for landmark in results.pose_landmarks.landmark:
            keypoints.append({
                "x": landmark.x,
                "y": landmark.y,
                "z": landmark.z,
                "visibility": landmark.visibility
            })
    return keypoints

def save_keypoints_to_json(keypoints, output_folder, index,
↪output_format="json"):
    """Zapisz keypoints do pliku w formacie JSON lub CSV"""
    filename = f"keypoints_{index:04d}.{output_format}" a
    output_path = os.path.join(output_folder, filename)

    if output_format == "json":
        with open(output_path, 'w') as json_file:
            json.dump(keypoints, json_file, indent=4)
            print(f"Keypoints zapisane do: {output_path}")

    elif output_format == "csv":
        with open(output_path, 'w', newline='') as csv_file:
            writer = csv.writer(csv_file)
            for point in keypoints:
                writer.writerow(point)
            print(f"Keypoints zapisane do: {output_path}")

def process_image_and_save_keypoints(image_path, output_folder, index,
↪output_format="json"):
    """Wczytaj obraz, ekstraktuj keypoints i zapisz do pliku JSON lub CSV"""
```



```

image = cv2.imread(image_path)

if image is None:
    print(f"Nie udało się wczytać obrazu z {image_path}")
    return

keypoints = extract_keypoints_from_image(image)

save_keypoints_to_json(keypoints, output_folder, index, output_format)

def load_images_from_folder(folder_path):
    """Wczytaj obrazy z folderu"""
    supported_formats = ['*.jpg', '*.png']
    image_files = []
    for fmt in supported_formats:
        image_files.extend(glob.glob(os.path.join(folder_path, fmt)))
    image_files.sort()
    return image_files

def process_folder_and_save_keypoints(images_folder, output_folder,
    ↪output_format="json"):
    """Przetwórz wszystkie obrazy w folderze i zapisz keypoints do plików JSON,
    ↪lub CSV"""
    image_files = load_images_from_folder(images_folder)

    for idx, image_path in enumerate(image_files, start=1):
        process_image_and_save_keypoints(image_path, output_folder, idx,
    ↪output_format)

class KeypointDataLoader:
    def __init__(self, data_path, file_format="json"):
        self.data_path = data_path
        self.file_format = file_format
        self.keypoints = self.load_keypoints()

    def load_keypoints(self):
        """Wczytaj keypoints z pliku JSON"""
        keypoints = []
        if self.file_format == "json":
            with open(self.data_path, 'r') as jsonfile:
                keypoints = json.load(jsonfile)
        return keypoints

    def visualize_keypoints_on_image(self, image):
        """Wizualizuj keypoints na obrazie"""

```

```

        for keypoint in self.keypoints:
            x, y = int(keypoint["x"] * image.shape[1]), int(keypoint["y"] *
↳image.shape[0])
            cv2.circle(image, (x, y), 5, (0, 255, 0), -1)
        return image

def main():
    # Ścieżki do folderów
    images_folder = "/content/images" # Folder z obrazami
    output_folder = "/content/keypoints" # Folder do zapisu plików JSON

    os.makedirs(output_folder, exist_ok=True)
    clear_folder(output_folder)
    process_folder_and_save_keypoints(images_folder, output_folder,
↳output_format="json")

    image_files = load_images_from_folder(images_folder)
    json_files = sorted(glob.glob(os.path.join(output_folder, "*.json")))

    for image_path, json_path in zip(image_files, json_files):
        loader = KeypointDataLoader(json_path, file_format="json")

        # Wczytaj obraz i wizualizuj keypoints
        image = cv2.imread(image_path)
        loader = KeypointDataLoader(json_path, file_format="json")
        img_with_keypoints = loader.visualize_keypoints_on_image(image)
        cv2.imshow(img_with_keypoints)

if __name__ == '__main__':
    main()

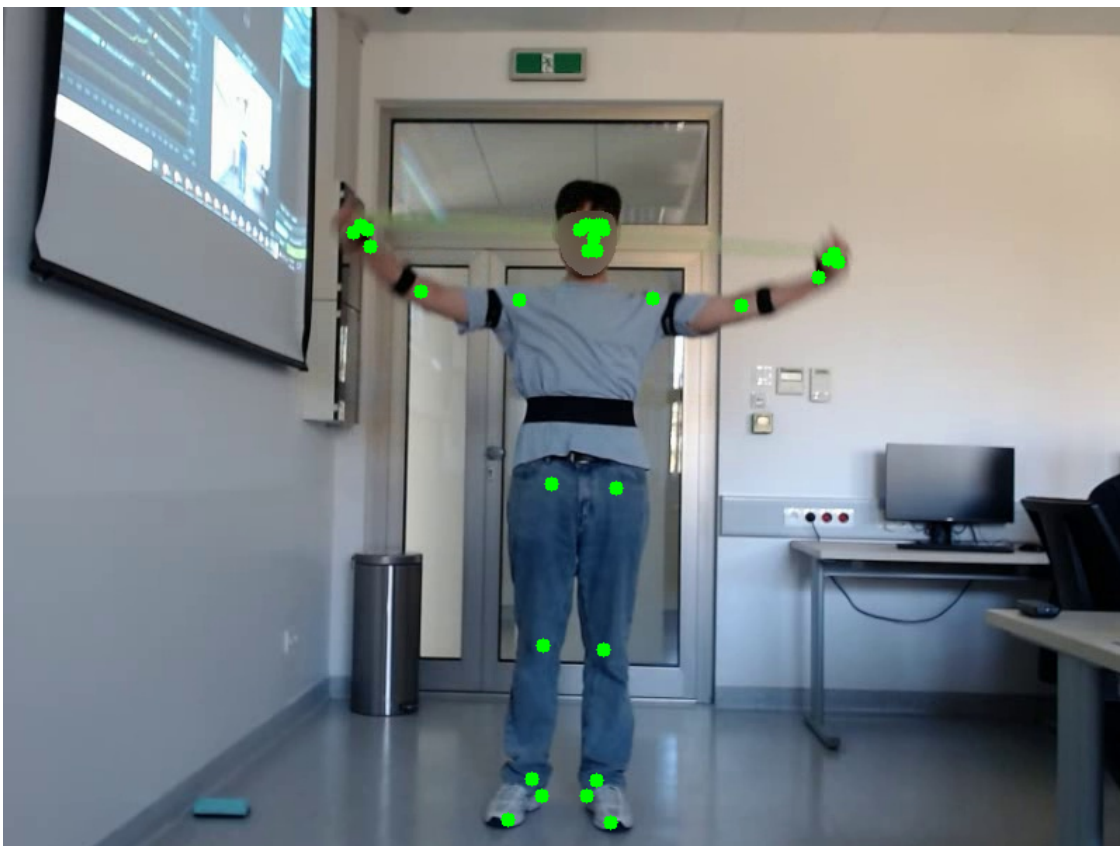
```

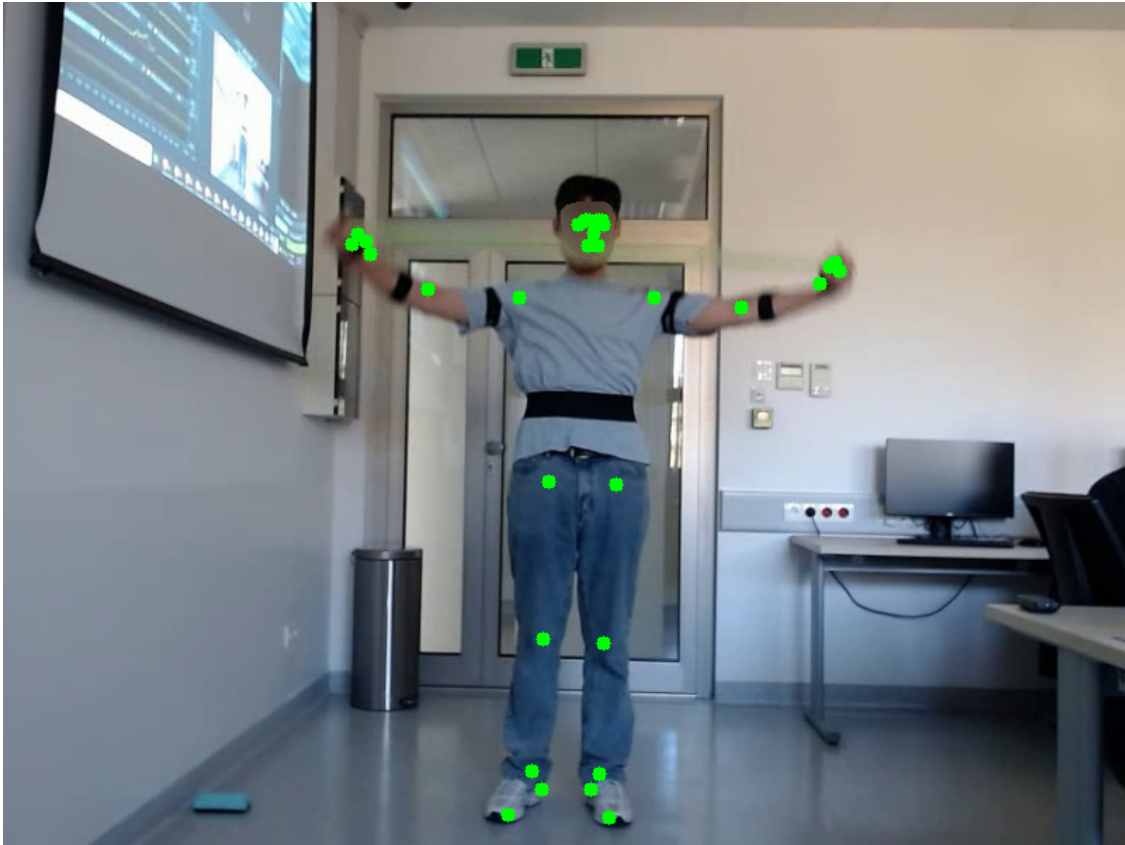
```

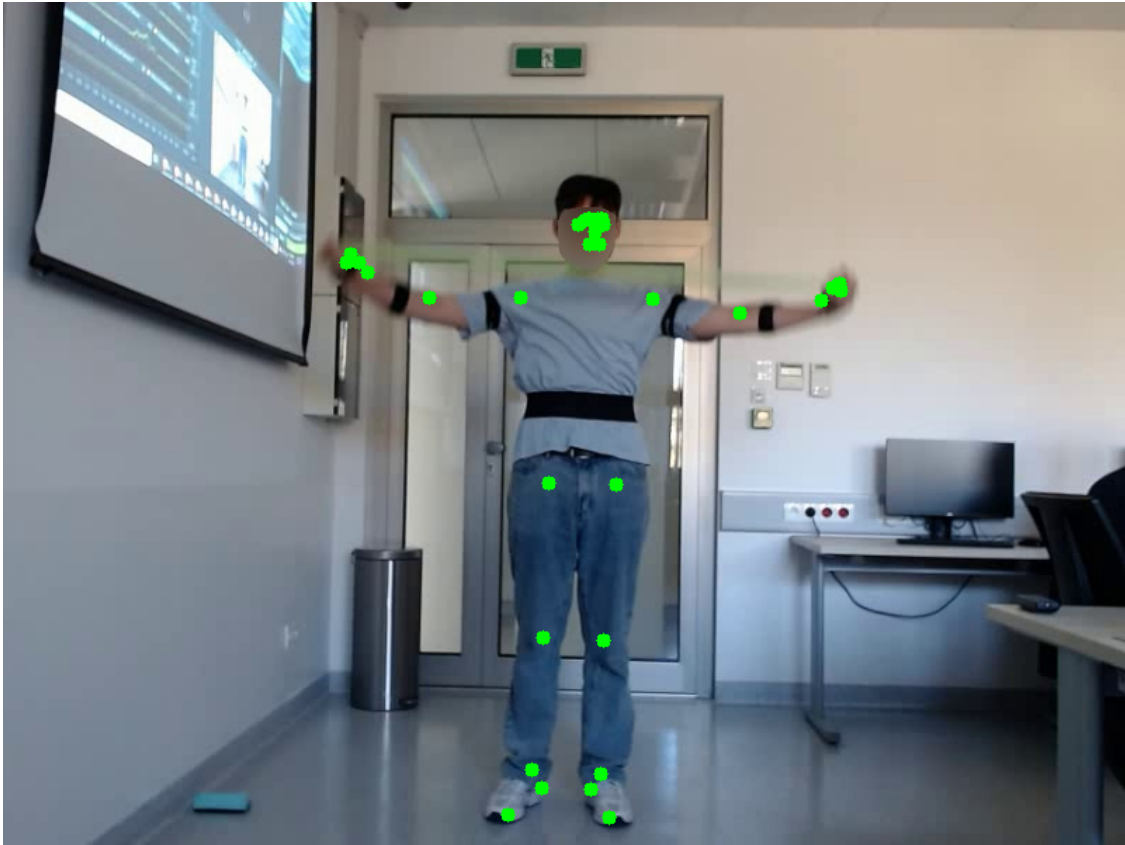
Keypoints zapisane do: /content/keypoints/keypoints_0001.json
Keypoints zapisane do: /content/keypoints/keypoints_0002.json
Keypoints zapisane do: /content/keypoints/keypoints_0003.json
Keypoints zapisane do: /content/keypoints/keypoints_0004.json
Keypoints zapisane do: /content/keypoints/keypoints_0005.json
Keypoints zapisane do: /content/keypoints/keypoints_0006.json
Keypoints zapisane do: /content/keypoints/keypoints_0007.json
Keypoints zapisane do: /content/keypoints/keypoints_0008.json
Keypoints zapisane do: /content/keypoints/keypoints_0009.json
Keypoints zapisane do: /content/keypoints/keypoints_0010.json

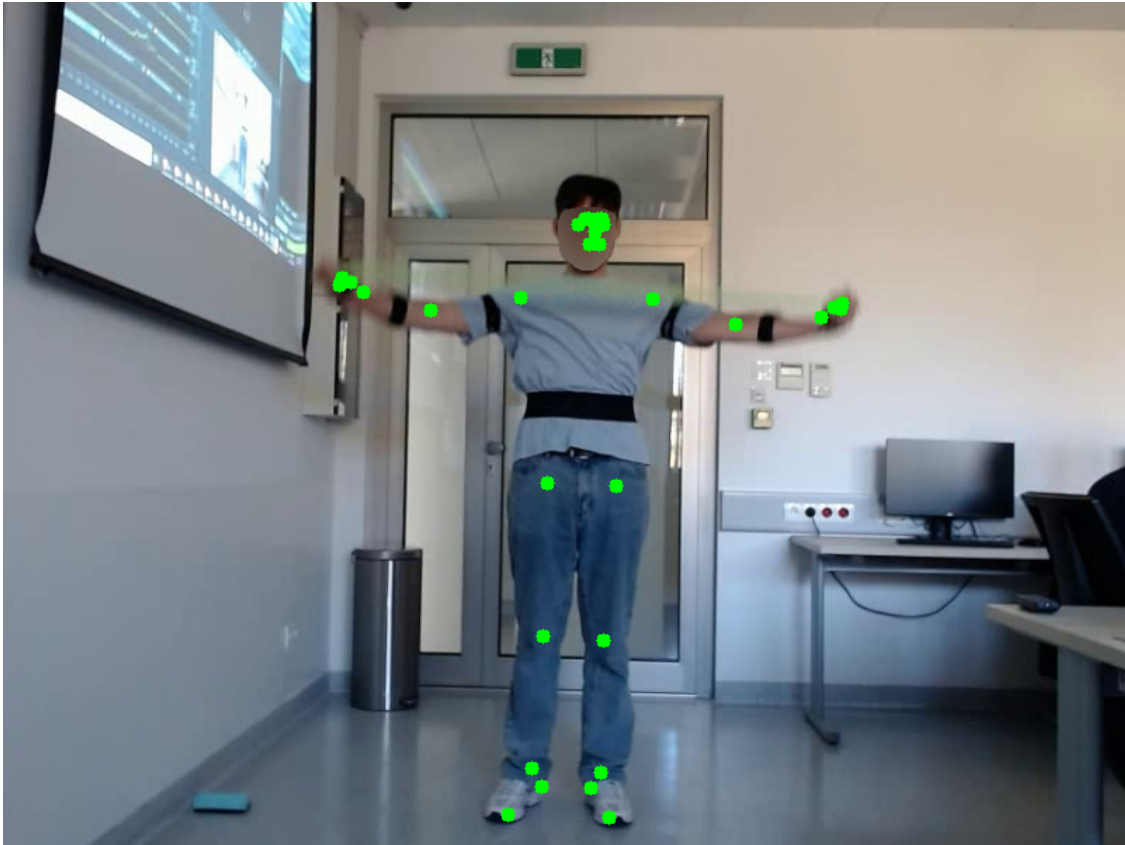
```

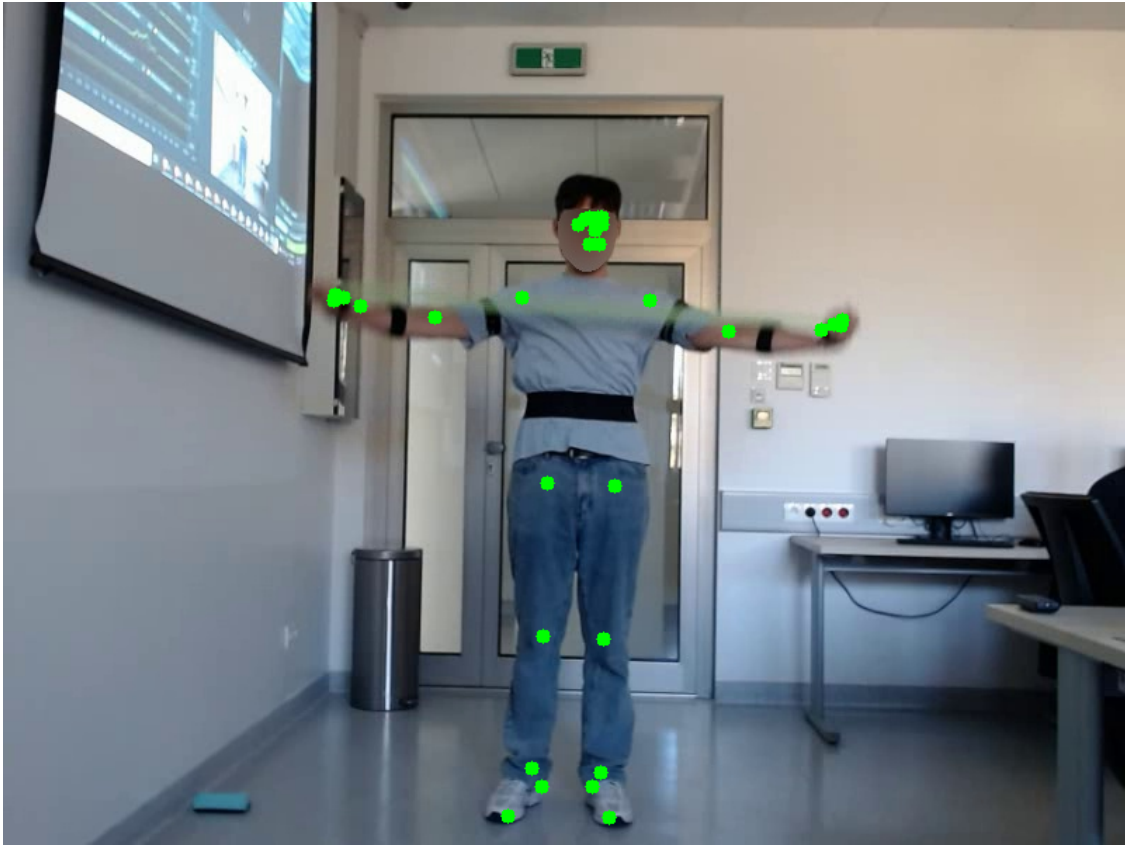
Keypoints zapisane do: /content/keypoints/keypoints_0011.json
Keypoints zapisane do: /content/keypoints/keypoints_0012.json
Keypoints zapisane do: /content/keypoints/keypoints_0013.json
Keypoints zapisane do: /content/keypoints/keypoints_0014.json
Keypoints zapisane do: /content/keypoints/keypoints_0015.json
Keypoints zapisane do: /content/keypoints/keypoints_0016.json
Keypoints zapisane do: /content/keypoints/keypoints_0017.json
Keypoints zapisane do: /content/keypoints/keypoints_0018.json
Keypoints zapisane do: /content/keypoints/keypoints_0019.json
Keypoints zapisane do: /content/keypoints/keypoints_0020.json
Keypoints zapisane do: /content/keypoints/keypoints_0021.json
Keypoints zapisane do: /content/keypoints/keypoints_0022.json
Keypoints zapisane do: /content/keypoints/keypoints_0023.json
Keypoints zapisane do: /content/keypoints/keypoints_0024.json
Keypoints zapisane do: /content/keypoints/keypoints_0025.json
Keypoints zapisane do: /content/keypoints/keypoints_0026.json
Keypoints zapisane do: /content/keypoints/keypoints_0027.json
Keypoints zapisane do: /content/keypoints/keypoints_0028.json

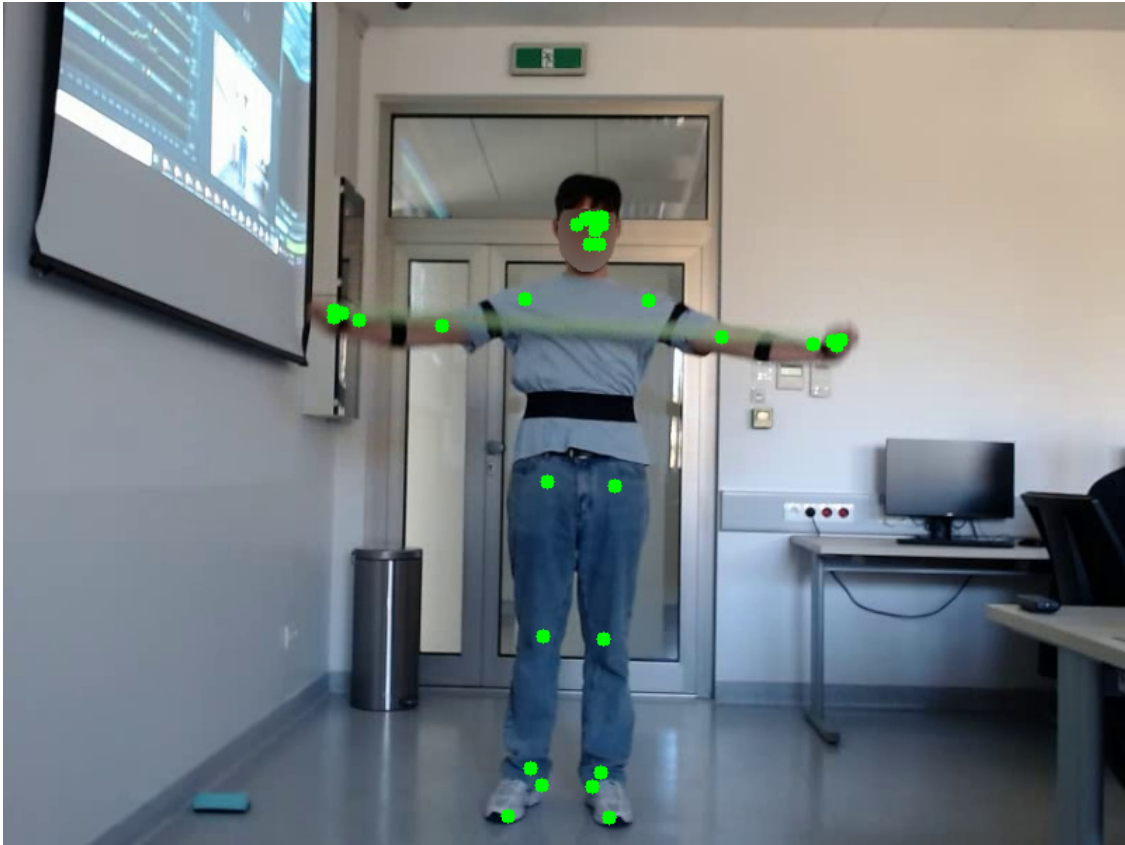


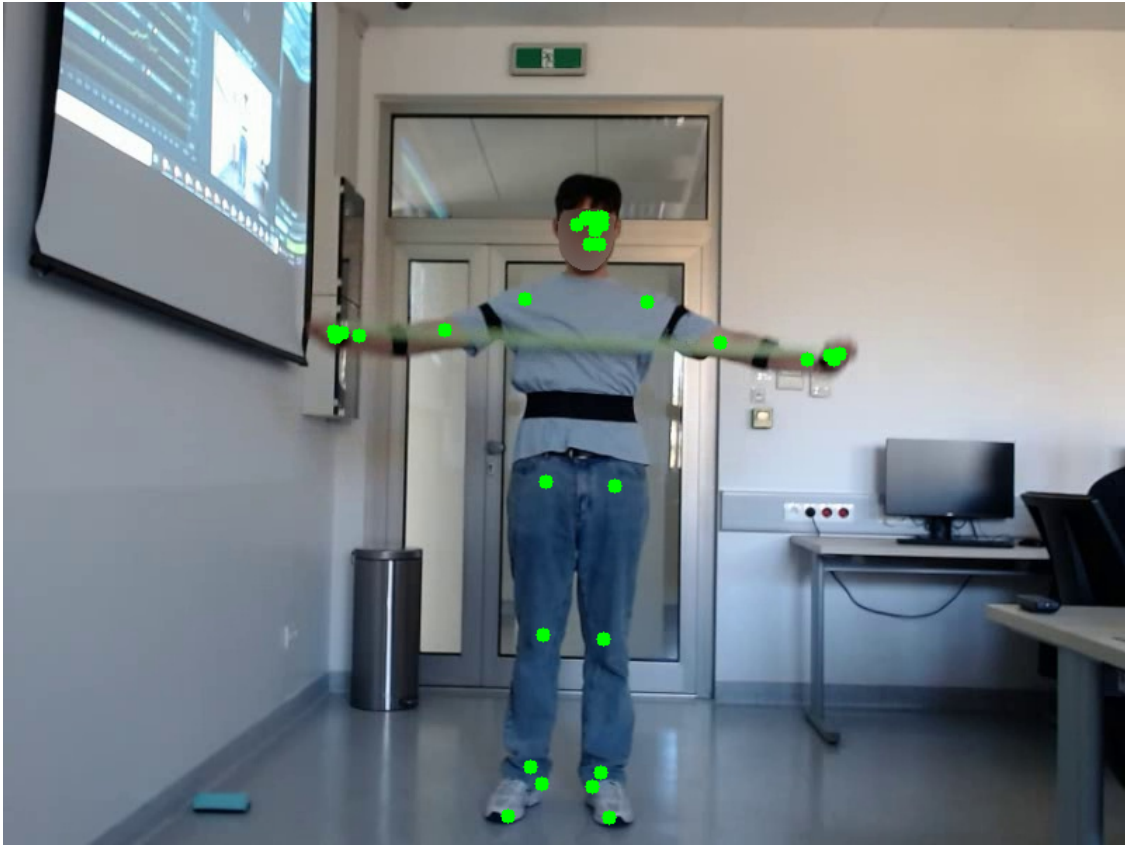


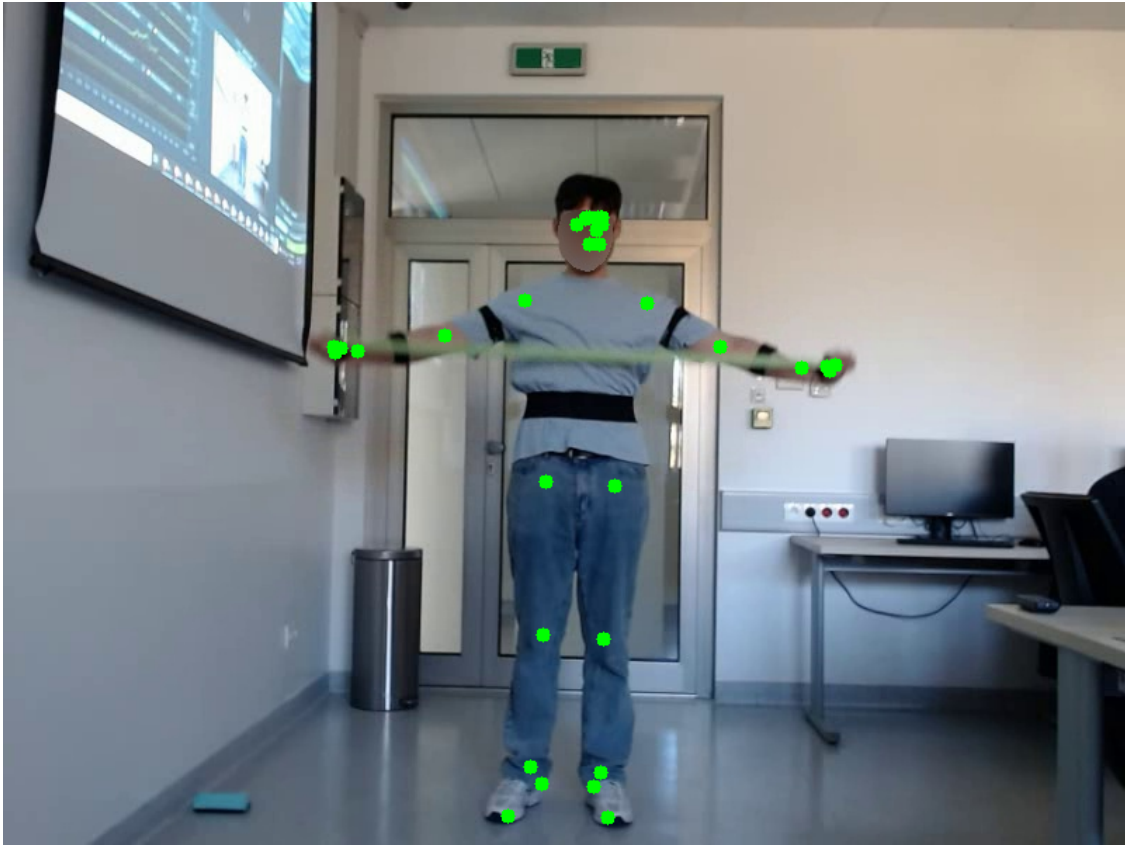


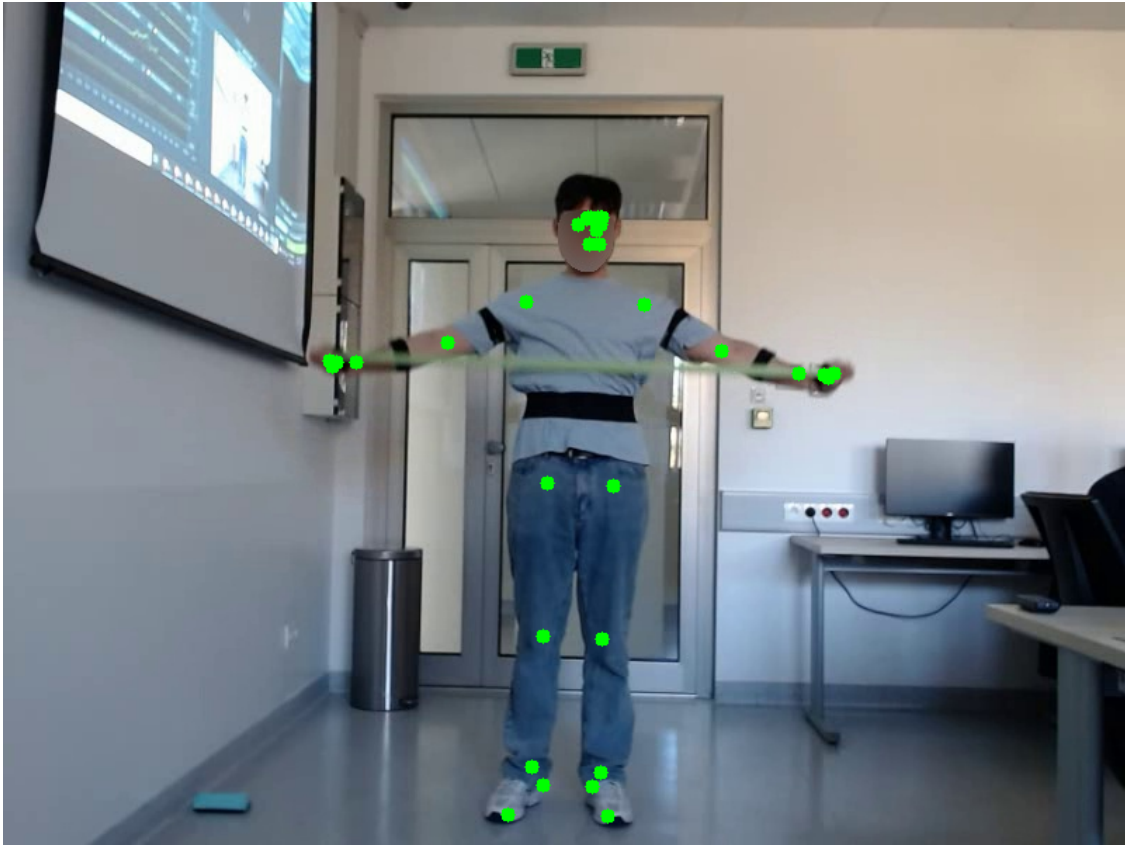


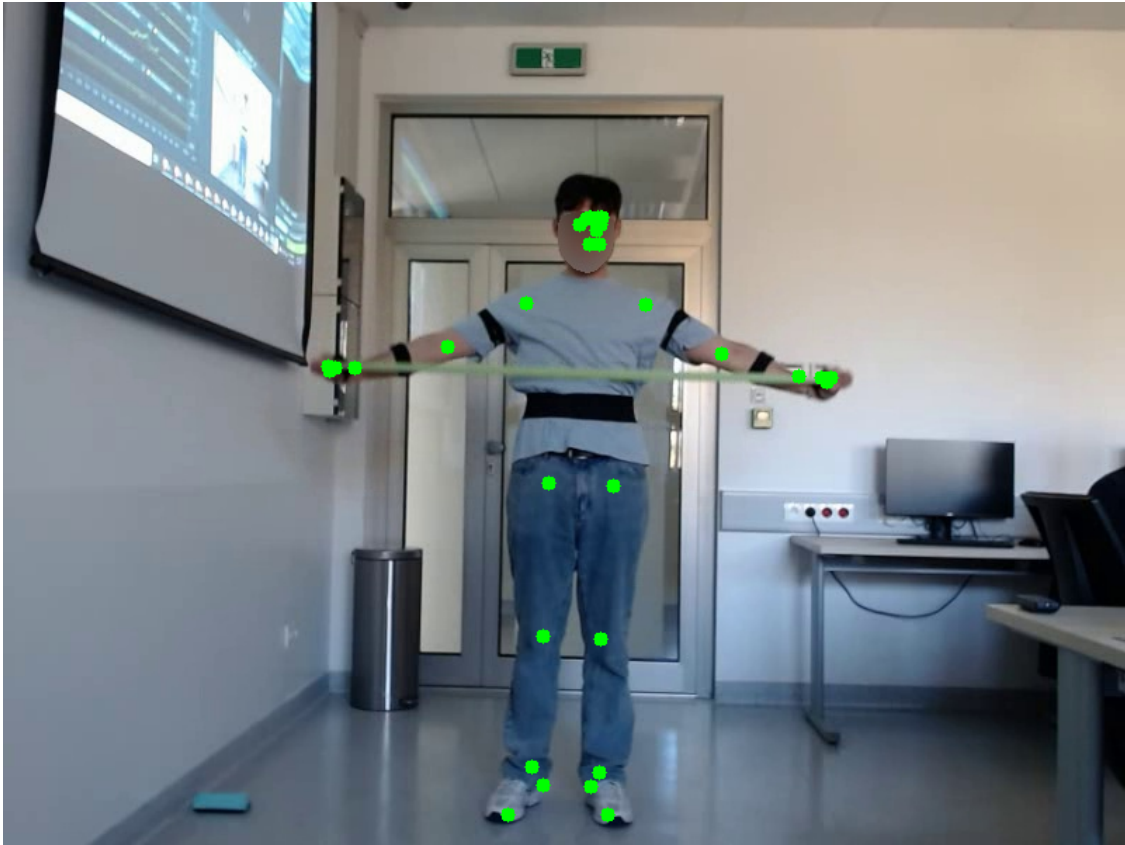


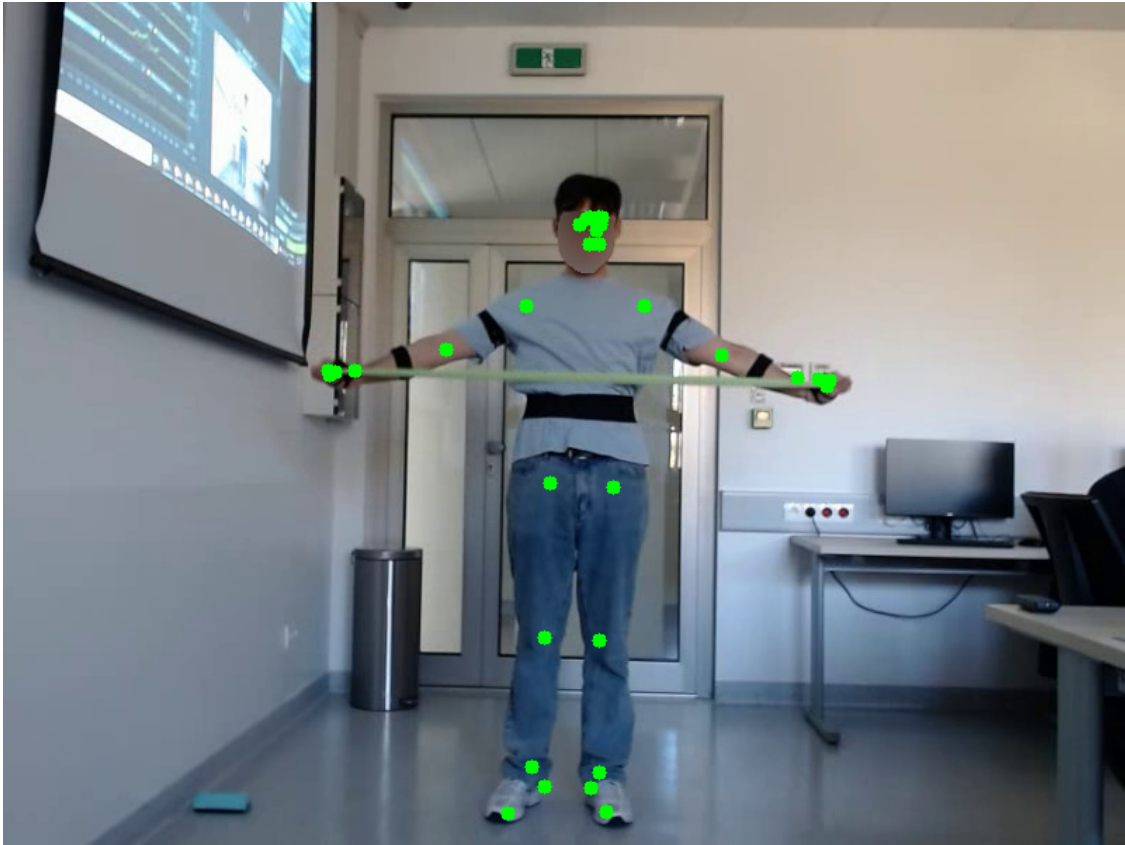


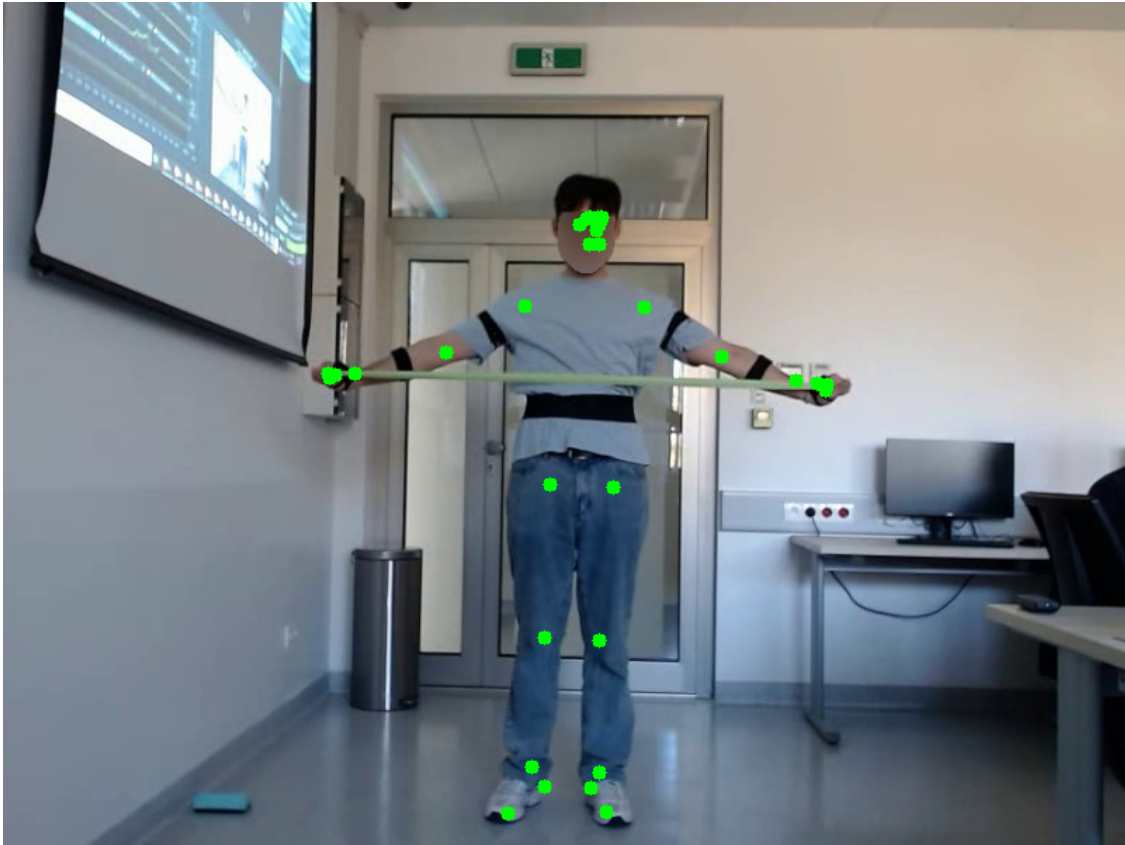


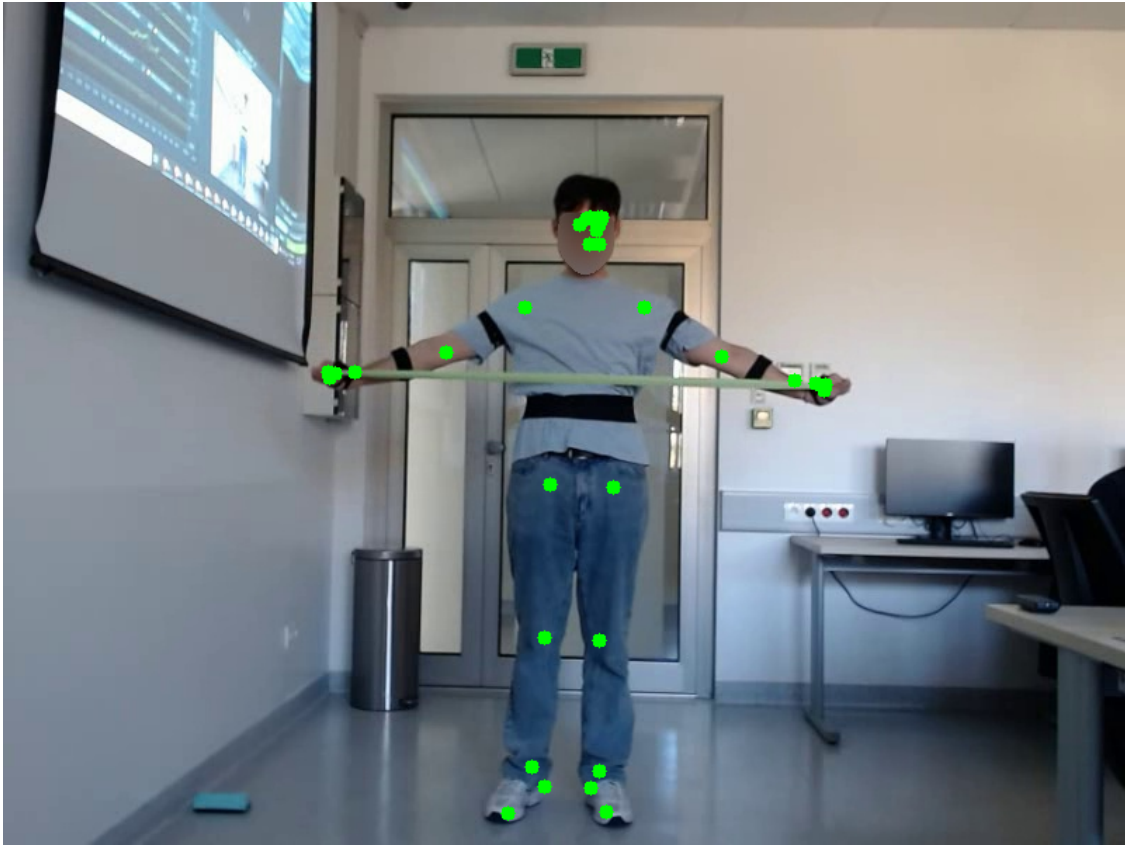


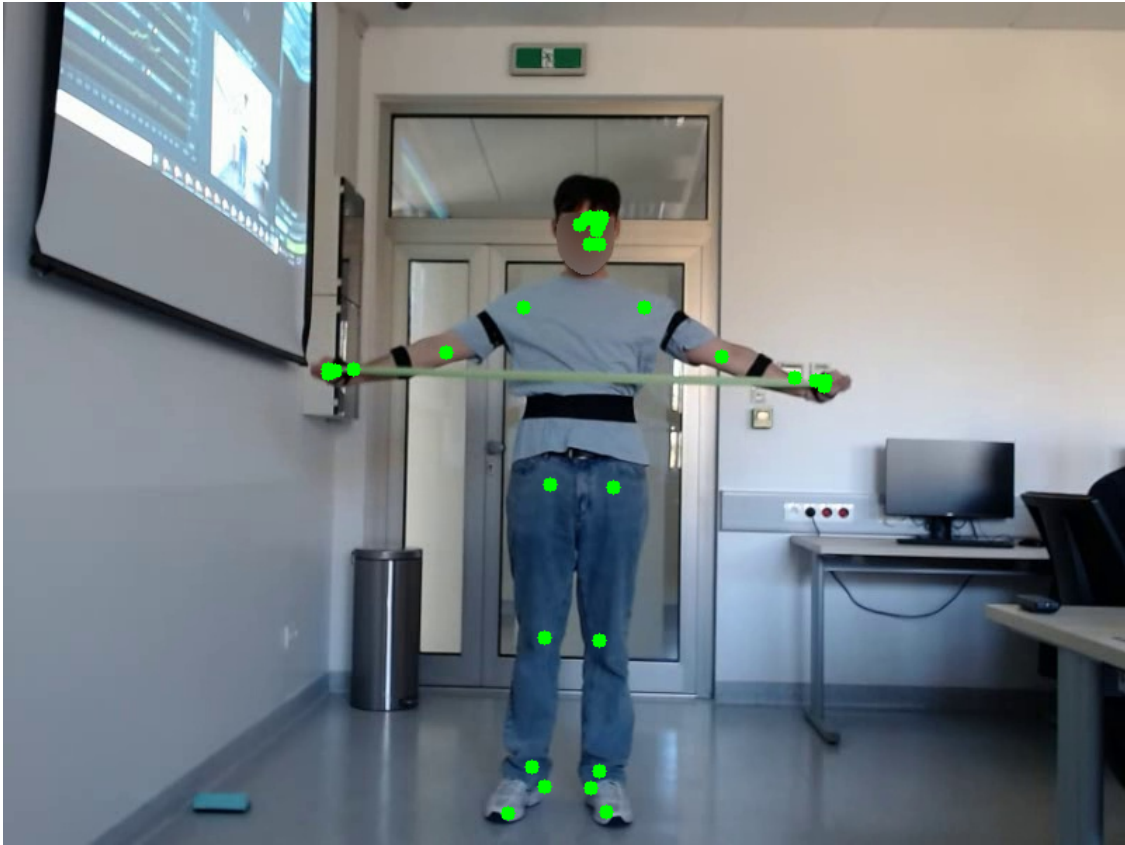


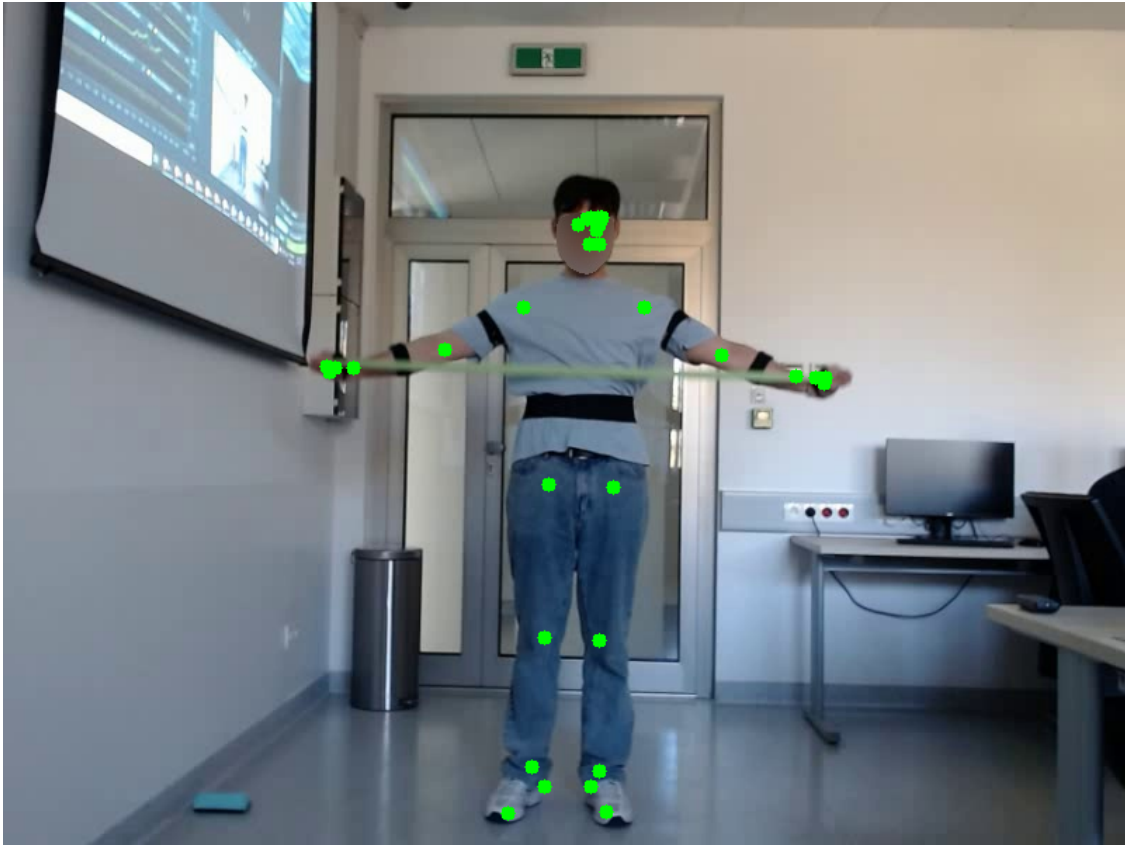


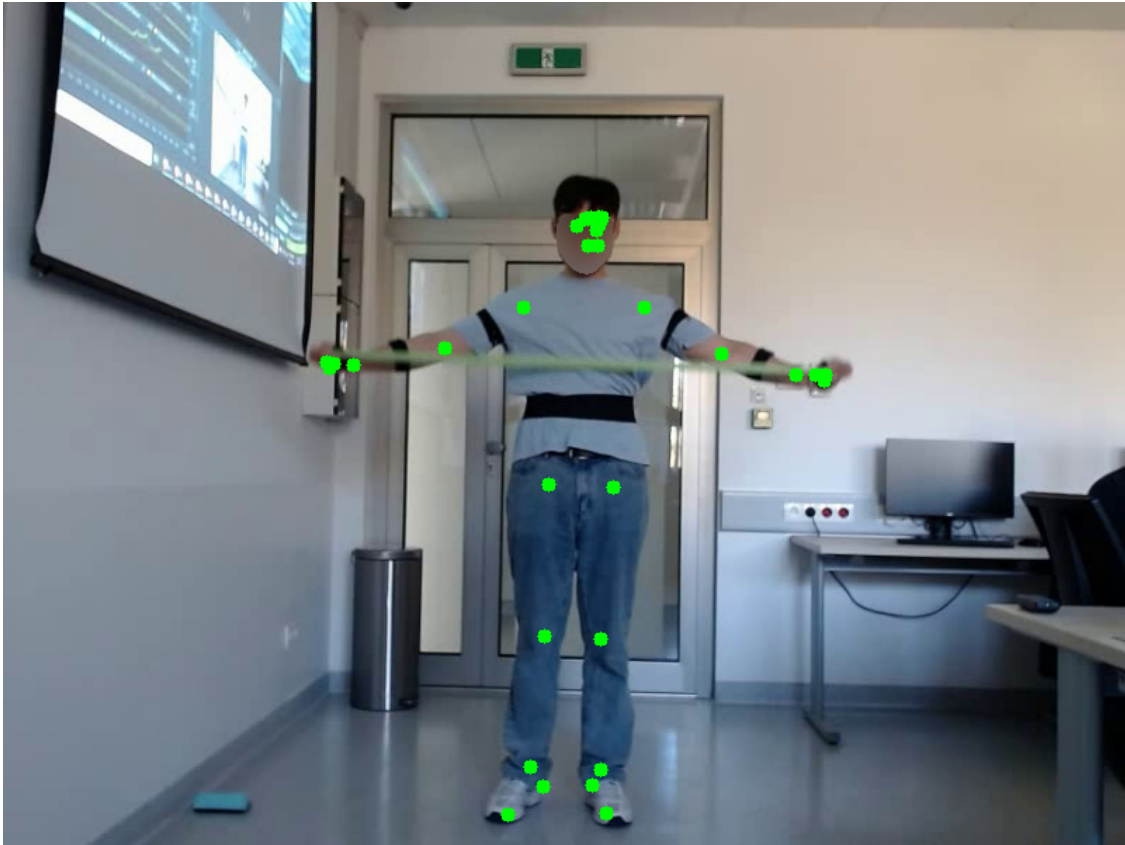


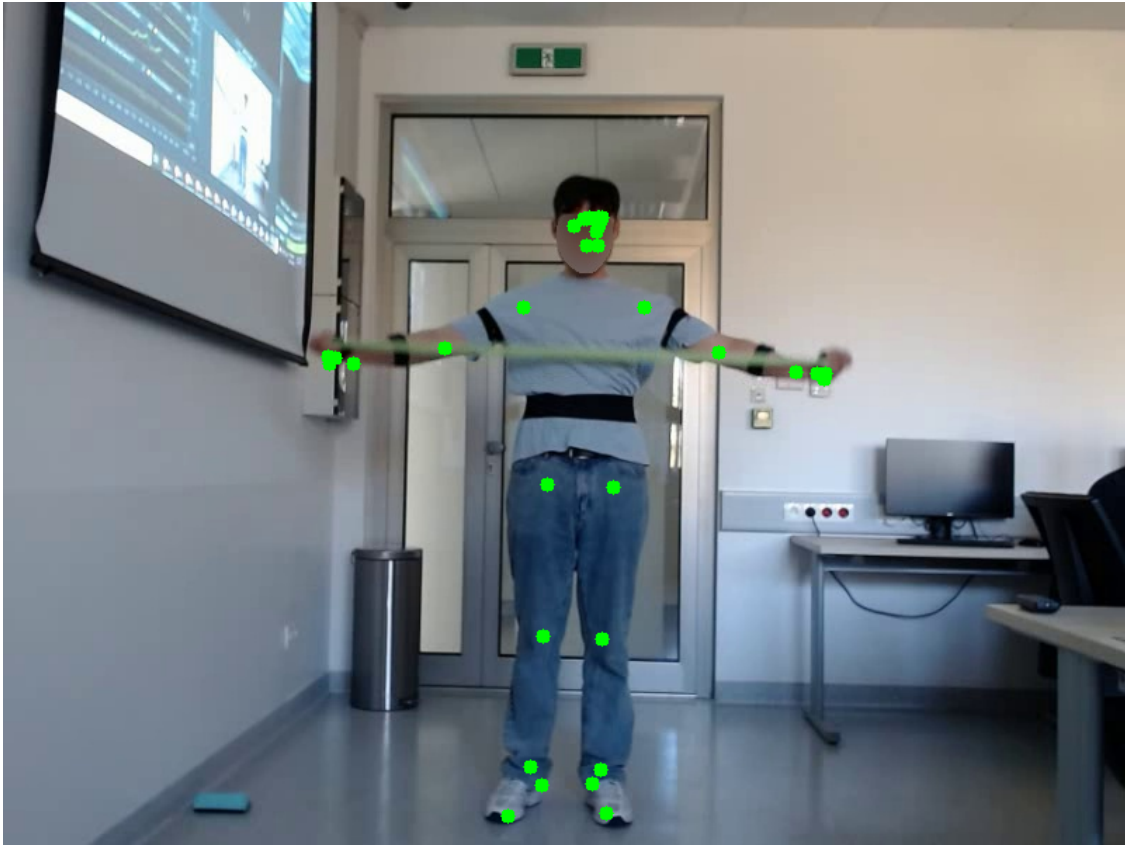


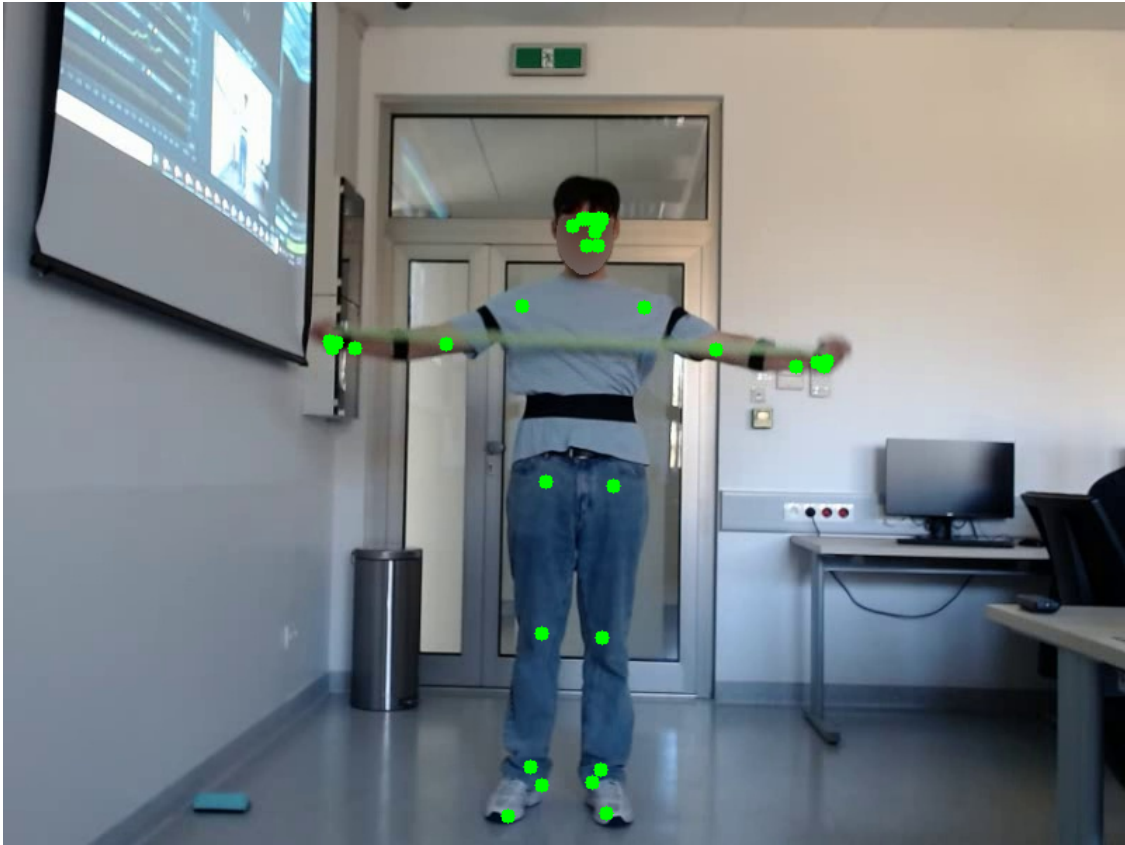


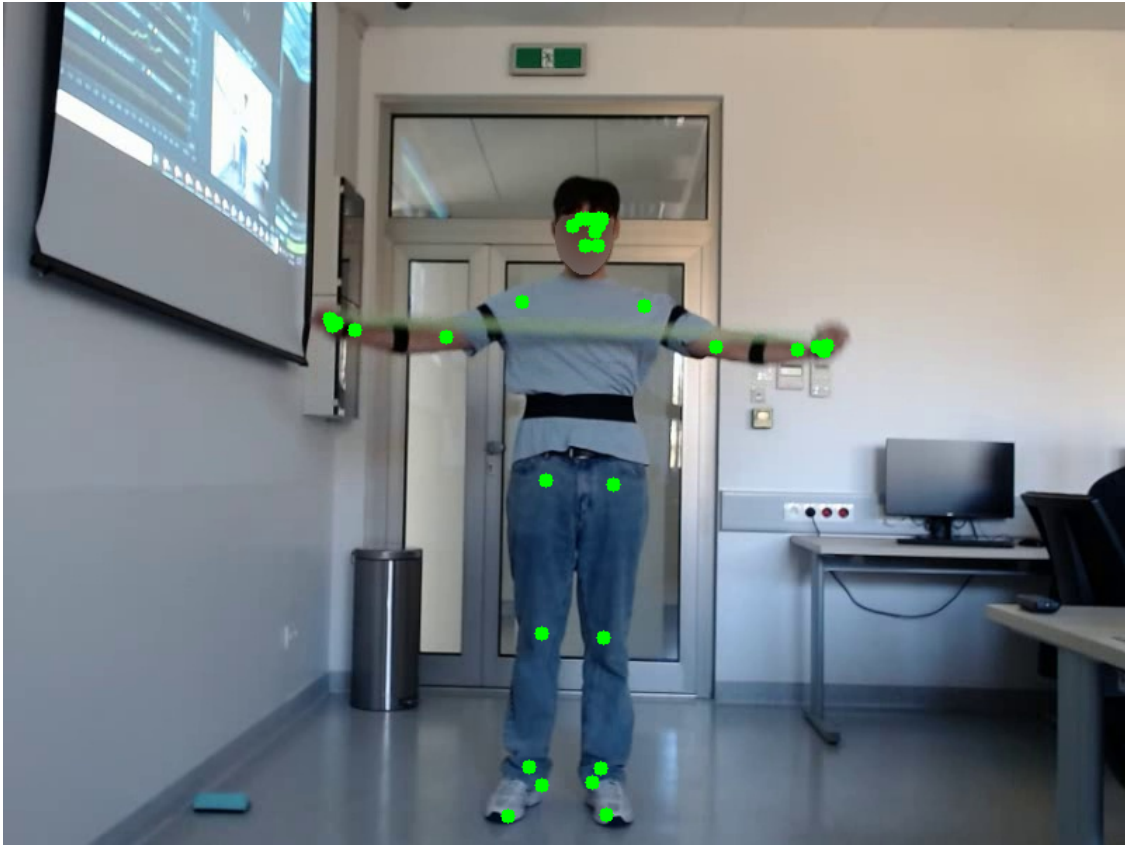


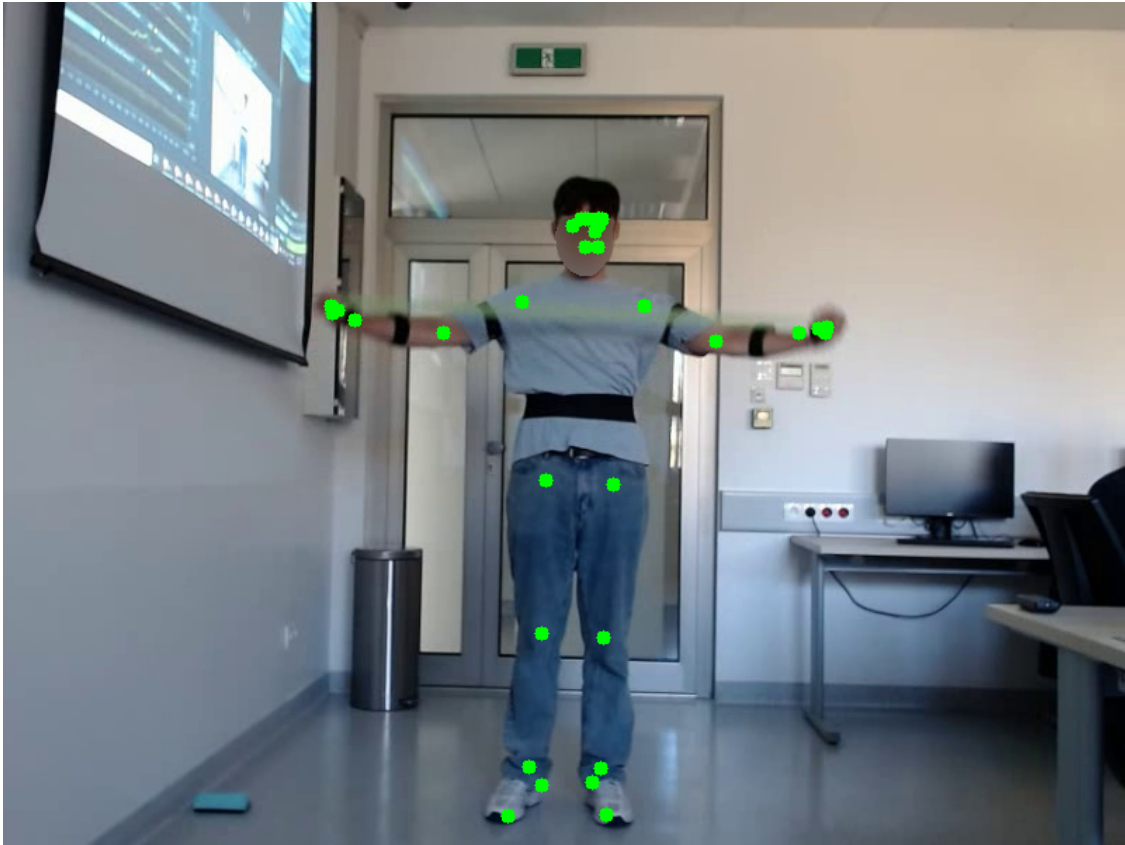


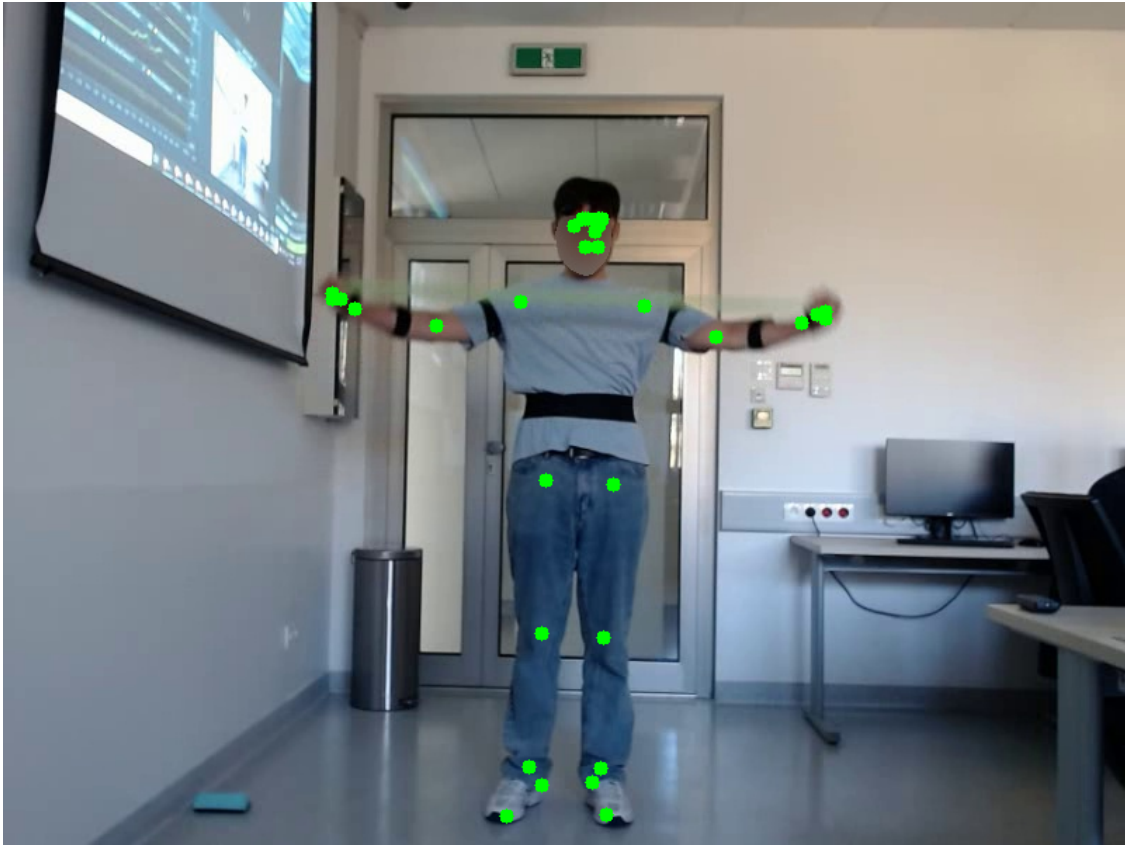


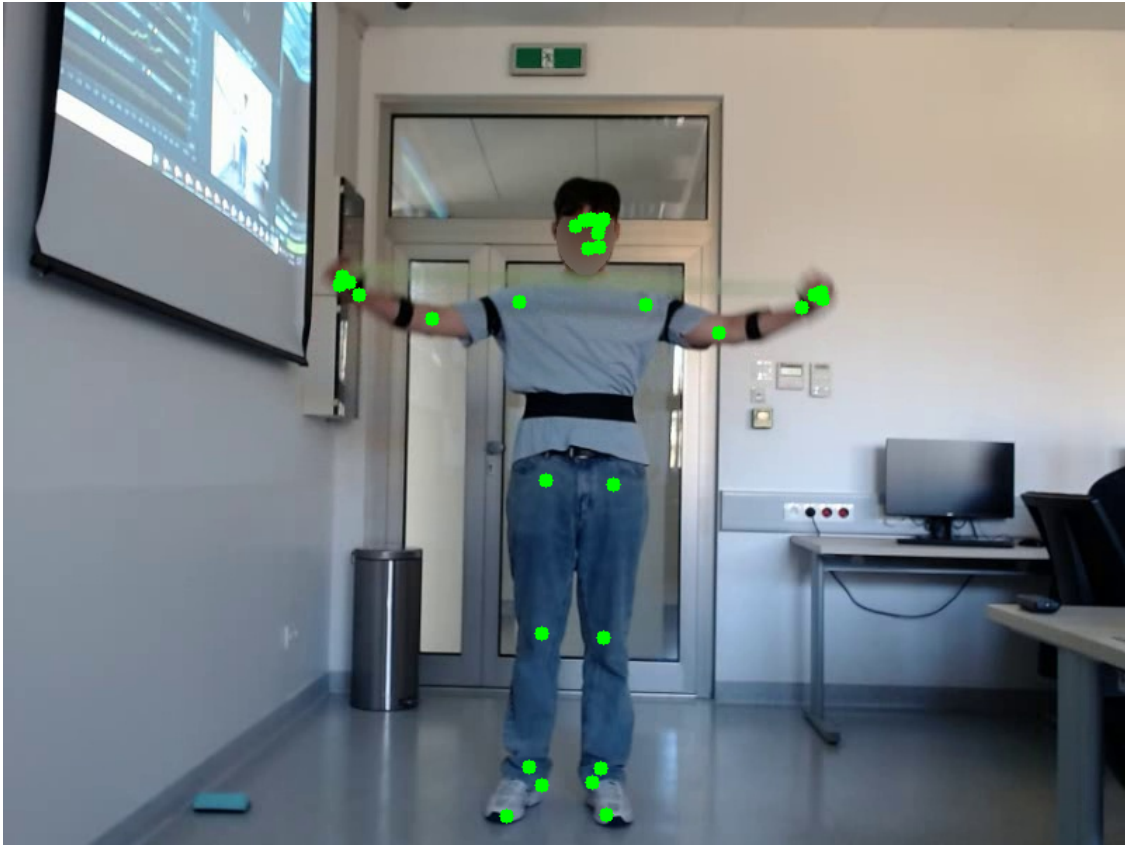


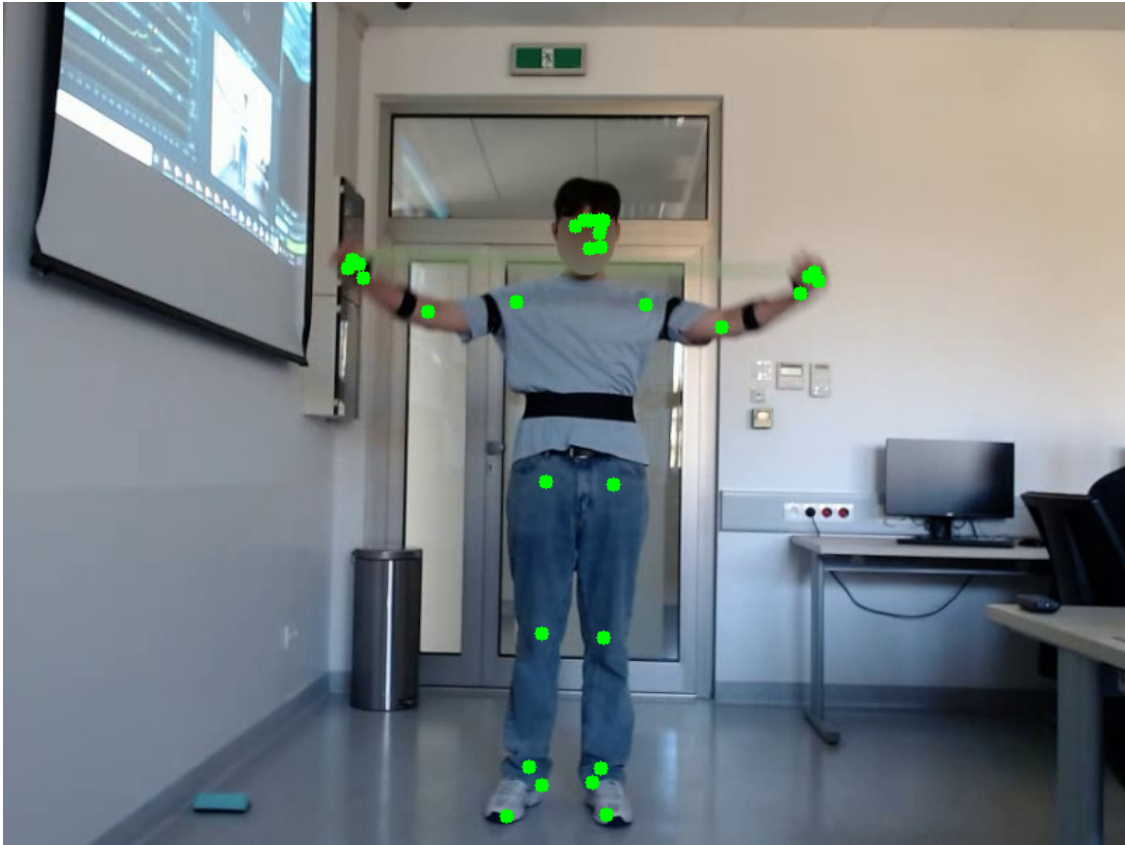


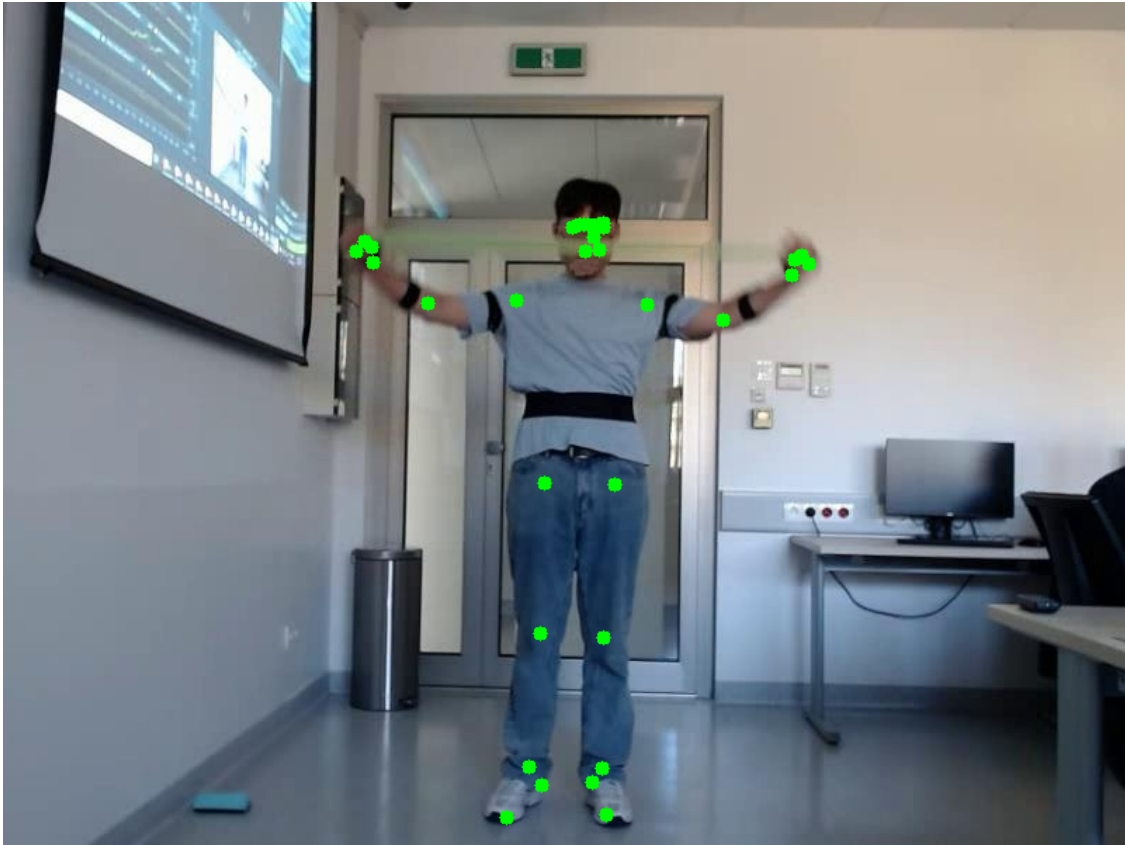


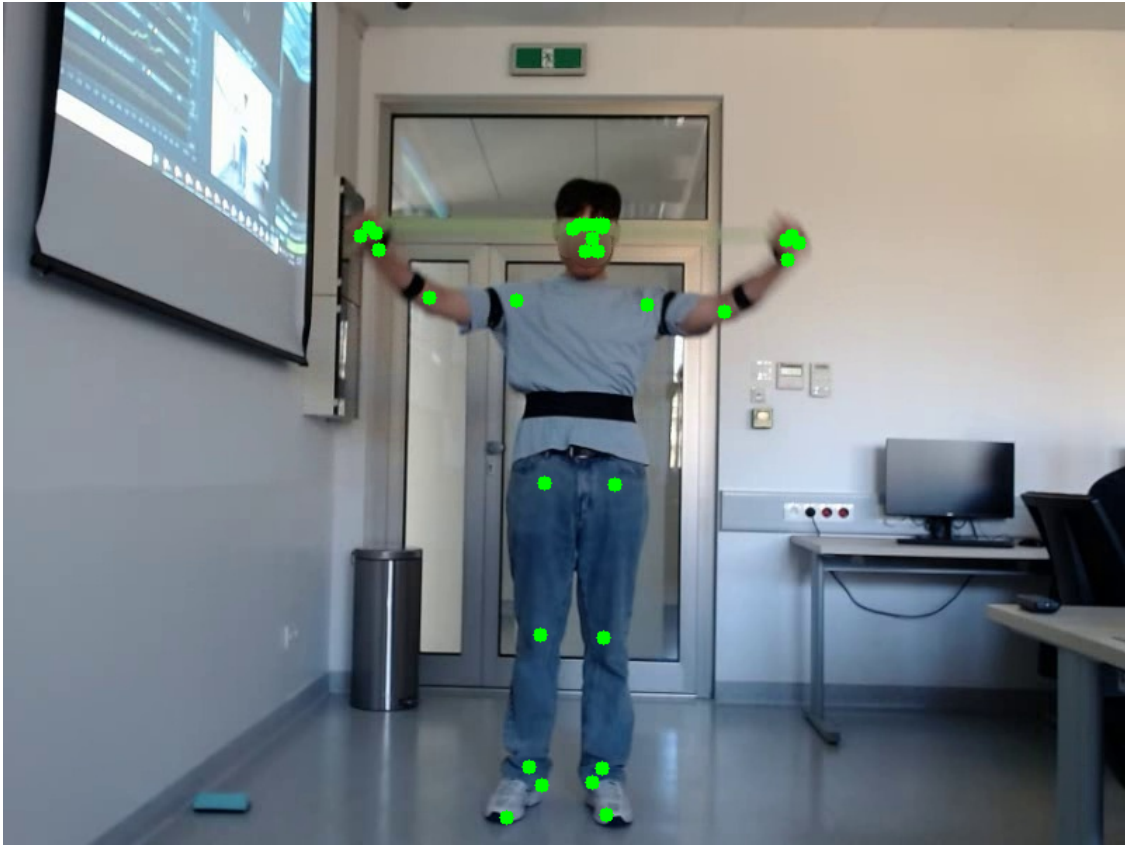


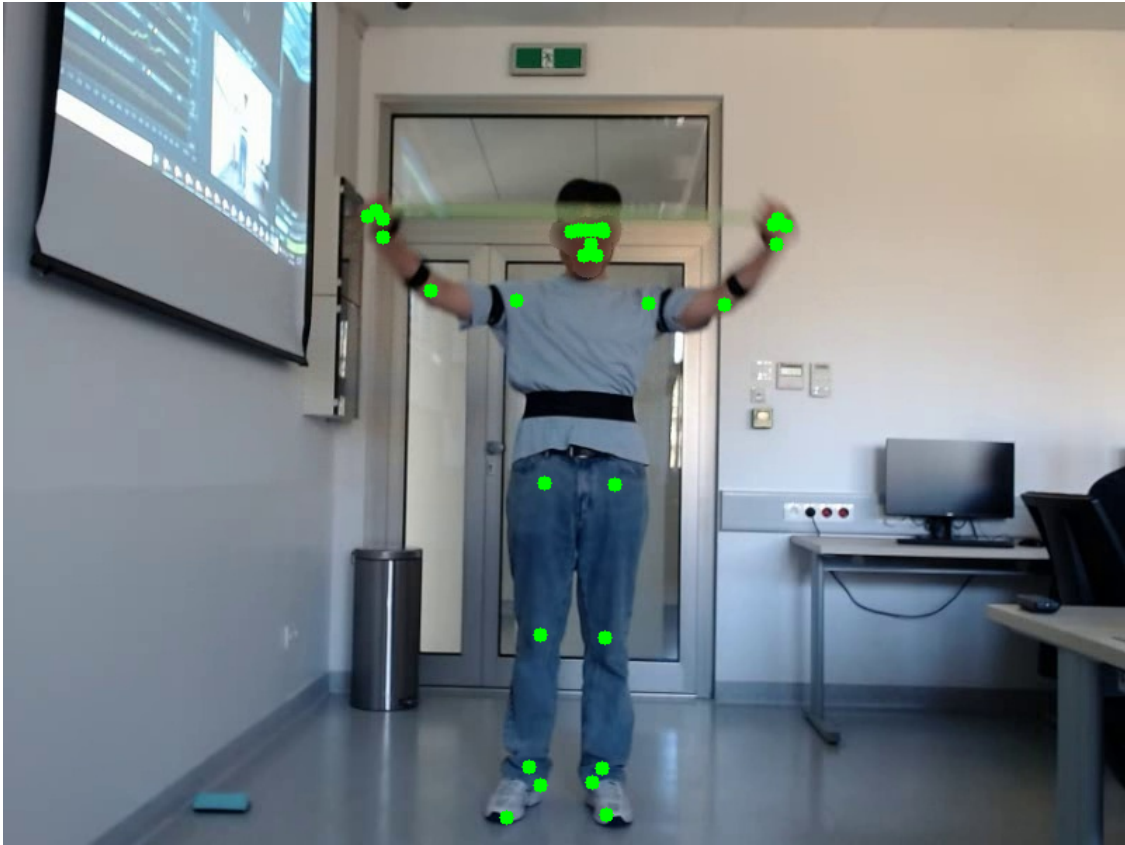


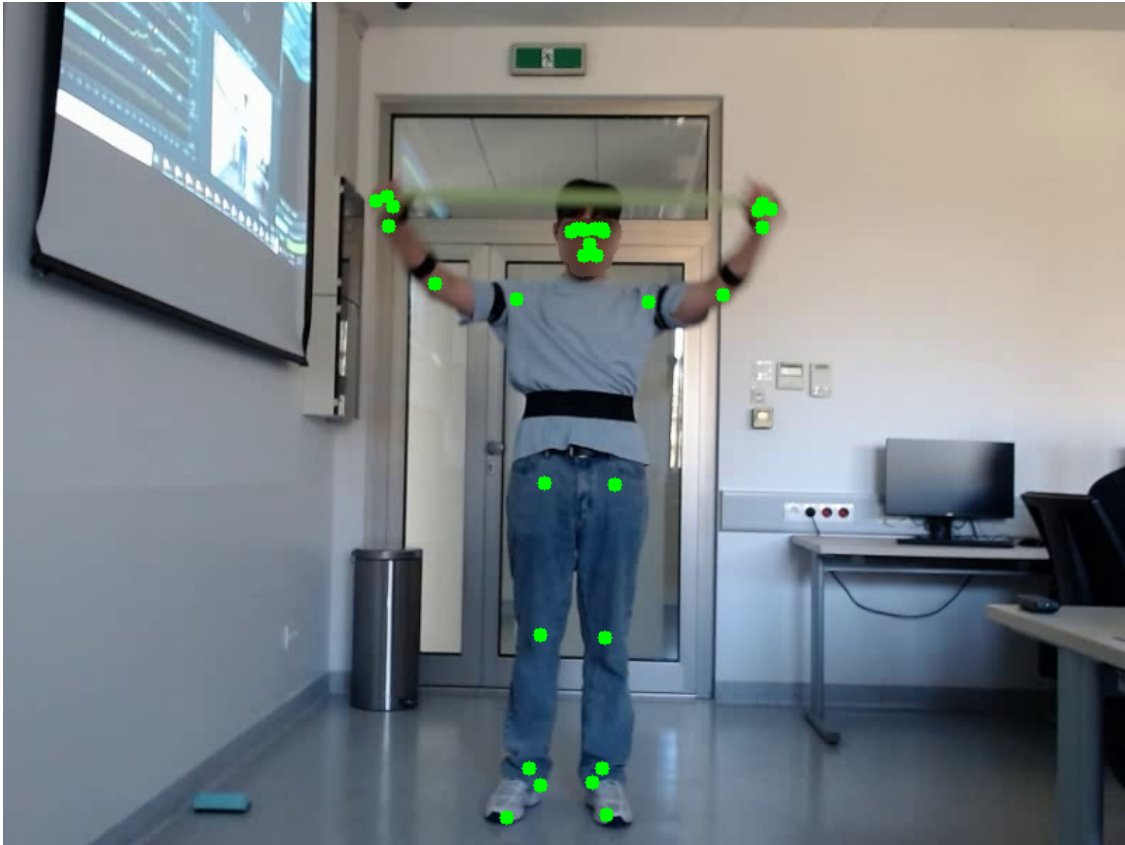


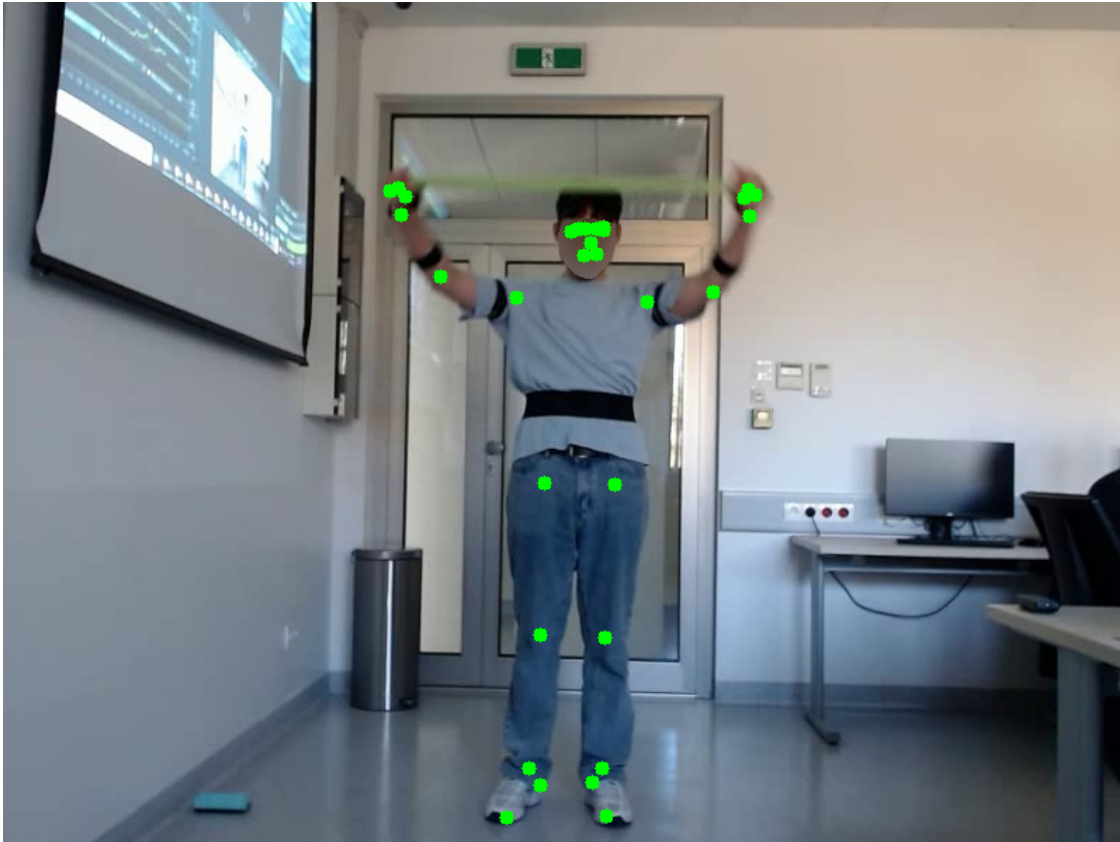












```
[ ]: import random

def augment_keypoints(keypoints, max_shift=10, scale_range=(0.8, 1.2)):
    """Augmentacja danych szkieletowych: przesunięcie i skalowanie (z
    uwzględnieniem struktury z 'x', 'y', 'z', 'visibility')"""

    shift_x = random.uniform(-max_shift, max_shift)
    shift_y = random.uniform(-max_shift, max_shift)

    # Skalowanie punktów w przestrzeni 2D
    scale = random.uniform(scale_range[0], scale_range[1])

    augmented_keypoints = []

    for point in keypoints:
        x = point['x']
        y = point['y']
        z = point['z']
        visibility = point['visibility']

        scale = random.uniform(-0.08, 0.08)
```

```

        augmented_x = (x * (1 + scale))
        augmented_y = (y * (1 + scale))
        augmented_z = z * (1 + scale)

        augmented_keypoints.append({
            'x': augmented_x,
            'y': augmented_y,
            'z': augmented_z,
            'visibility': visibility
        })

    return augmented_keypoints

def load_keypoints_from_json(json_path):
    """Wczytaj keypoints z pliku JSON"""
    with open(json_path, 'r') as file:
        keypoints = json.load(file)
    return keypoints

def save_keypoints_to_json(keypoints, output_folder, filename):
    """Zapisz zaugmentedowane keypoints do pliku JSON"""
    output_path = os.path.join(output_folder, filename)
    with open(output_path, 'w') as json_file:
        json.dump(keypoints, json_file, indent=4)
    print(f"Zapisano zaugmentedowane keypoints do: {output_path}")

def augment_all_keypoints_in_folder(input_folder, output_folder, max_shift=10,
    ↪scale_range=(0.8, 1.2)):
    """Augmentacja wszystkich keypoints w folderze i zapisanie wyników"""
    json_files = [f for f in os.listdir(input_folder) if f.endswith('.json')]

    os.makedirs(output_folder, exist_ok=True)
    clear_folder(output_folder)
    for json_file in json_files:
        json_path = os.path.join(input_folder, json_file)

        keypoints = load_keypoints_from_json(json_path)

        augmented_keypoints = augment_keypoints(keypoints, max_shift,
    ↪scale_range)

        augmented_json_file = f"augmented_{json_file}"
        save_keypoints_to_json(augmented_keypoints, output_folder,
    ↪augmented_json_file)

```

```

[ ]: # Ścieżki do folderów
input_folder = "/content/keypoints" # Folder z oryginalnymi plikami JSON

```



```

output_folder = "/content/augmented_keypoints" # Folder, gdzie zapiszemy
↳zaugmentedowane keypoints

augment_all_keypoints_in_folder(input_folder, output_folder, max_shift=10,
↳scale_range=(0.8, 1.2))

```

```

Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0024.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0004.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0016.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0022.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0002.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0027.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0018.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0012.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0015.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0011.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0003.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0019.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0026.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0025.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0023.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0008.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0013.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0017.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0010.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0028.json
Zapisano zaugmentedowane keypoints do:

```



```

/content/augmented_keypoints/augmented_keypoints_0014.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0001.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0009.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0005.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0007.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0021.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0020.json
Zapisano zaugmentedowane keypoints do:
/content/augmented_keypoints/augmented_keypoints_0006.json

```

```

[ ]: import cv2
import glob
import os
from google.colab.patches import cv2_imshow

images_folder = "/content/images" # Folder z obrazami
keypoints_folder = "/content/keypoints" # Folder z plikami JSON z keypoints
augmented_folder = "/content/augmented_keypoints" # Folder z plikami JSON z
↳ augmentedowanymi keypoints

image_files = sorted(glob.glob(os.path.join(images_folder, "*.jpg")))
keypoints_files = sorted(glob.glob(os.path.join(keypoints_folder, "*.json")))
augmented_files = sorted(glob.glob(os.path.join(augmented_folder, "*.json")))

for image_path, keypoints_path, augmented_path in zip(image_files,
↳ keypoints_files, augmented_files):
    image1 = cv2.imread(image_path)
    image2 = cv2.imread(image_path)

    original_loader = KeypointDataLoader(keypoints_path, file_format="json")
    original_keypoints = original_loader.keypoints

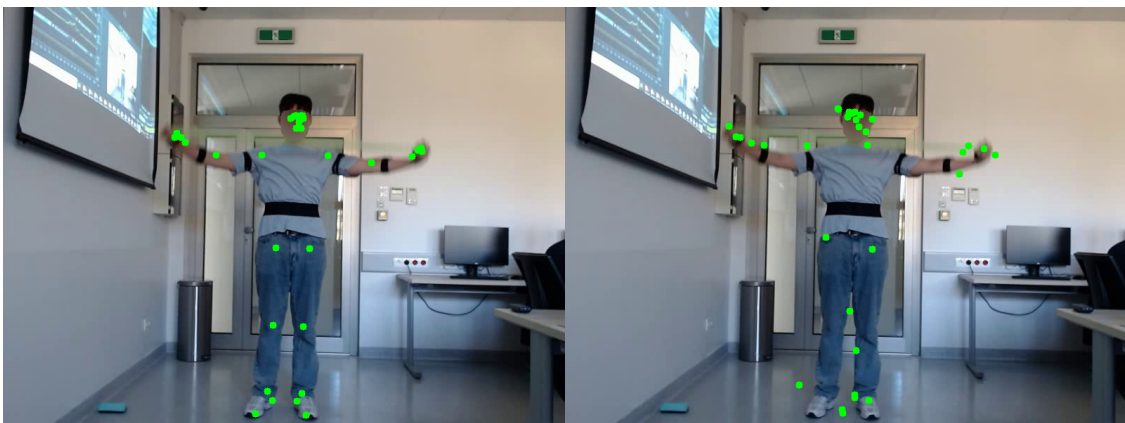
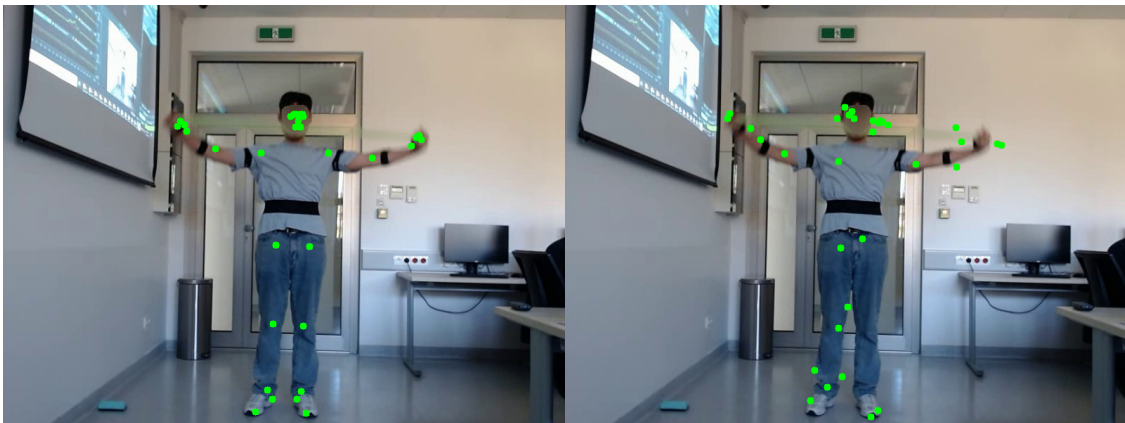
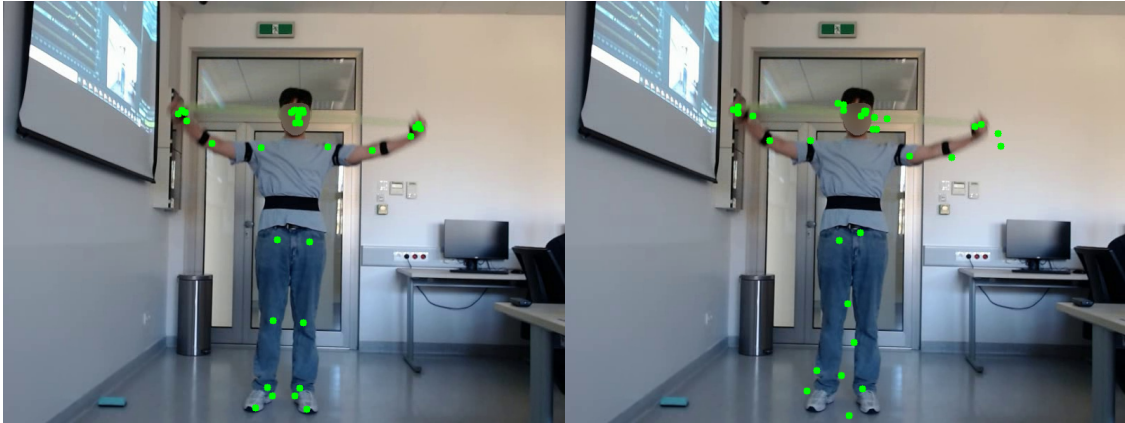
    augmented_loader = KeypointDataLoader(augmented_path, file_format="json")
    augmented_keypoints = augmented_loader.keypoints

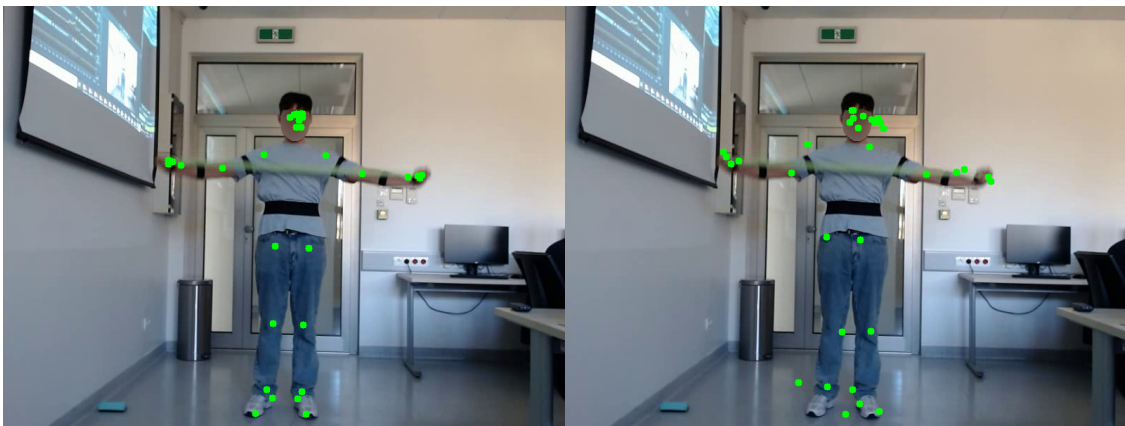
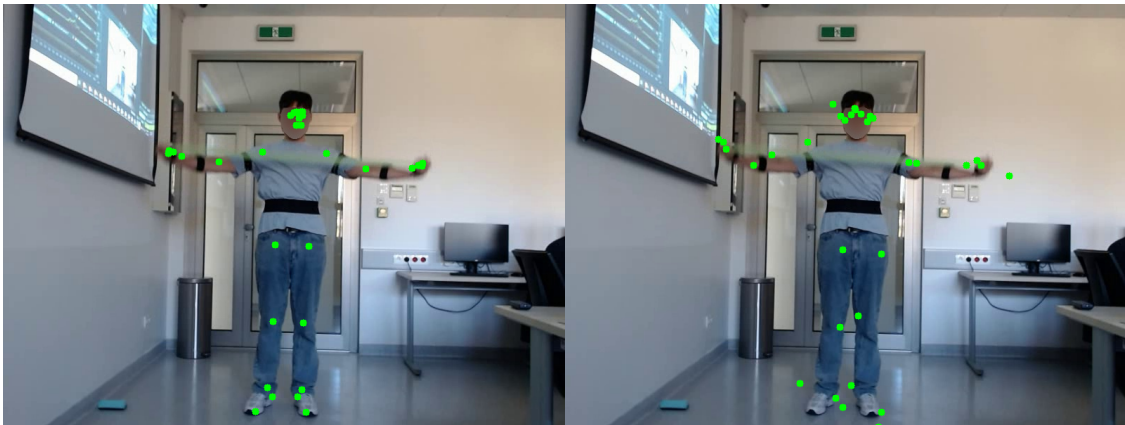
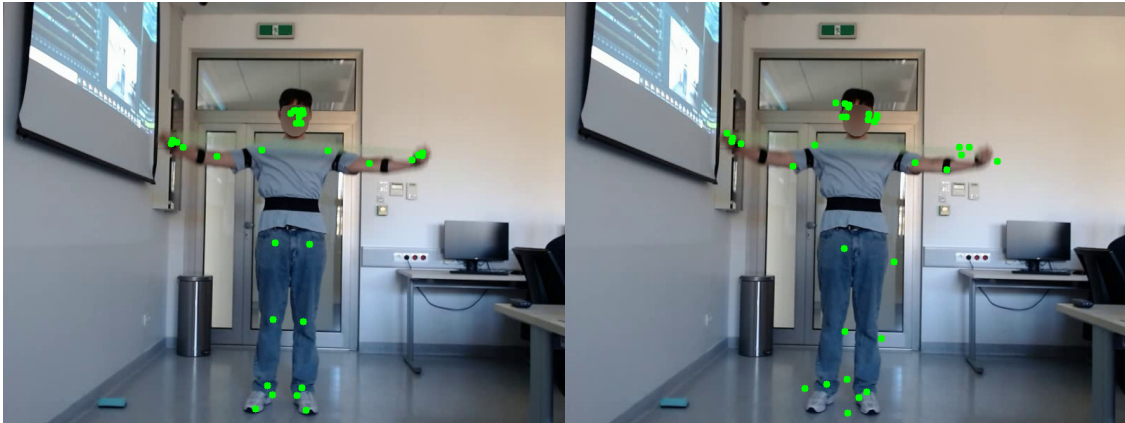
    original_image_with_keypoints = original_loader.
↳ visualize_keypoints_on_image(image1)

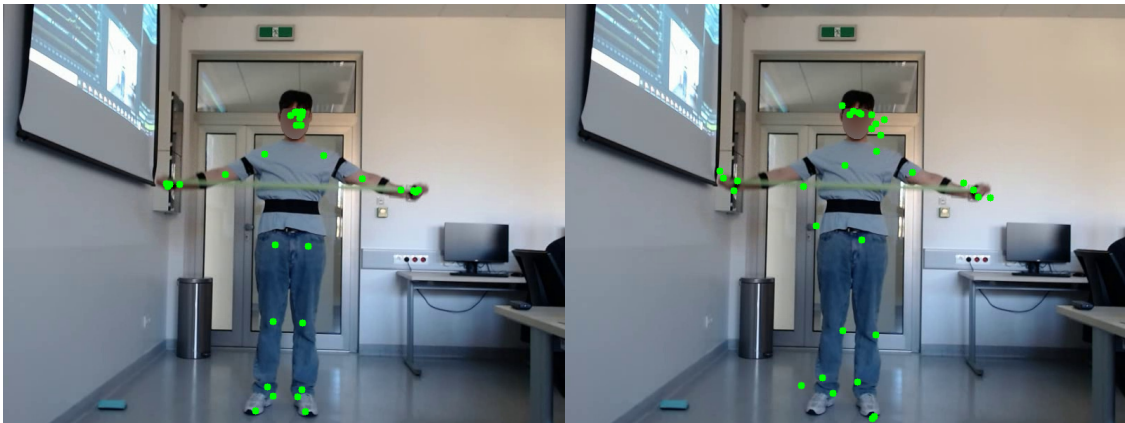
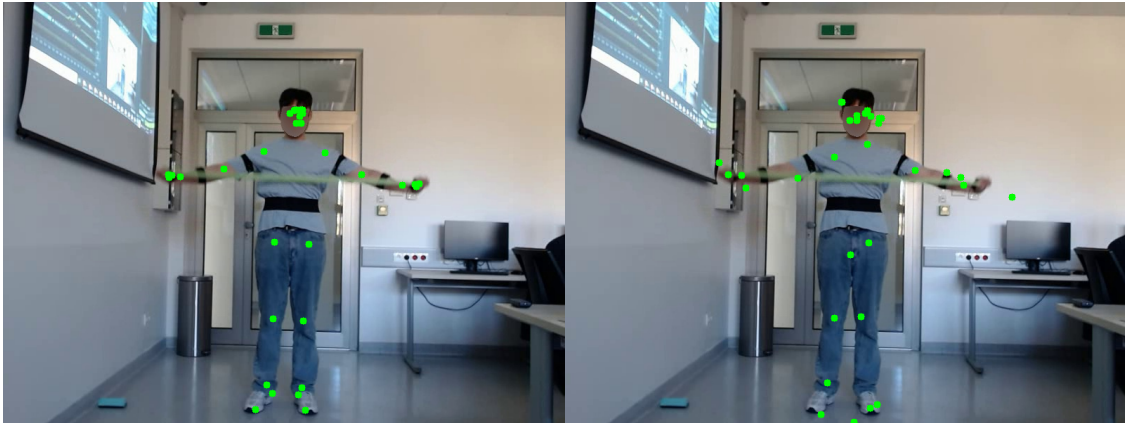
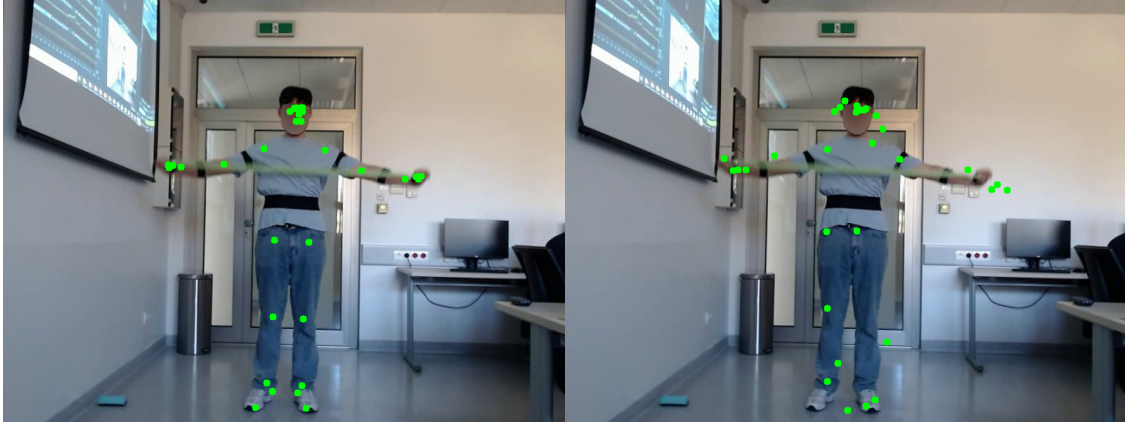
    augmented_loader.keypoints = augmented_keypoints
    augmented_image_with_keypoints = augmented_loader.
↳ visualize_keypoints_on_image(image2)

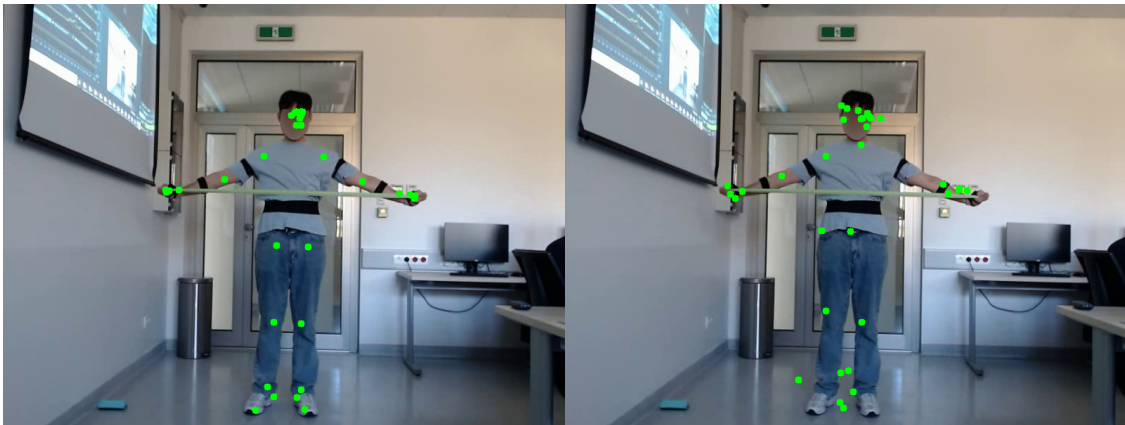
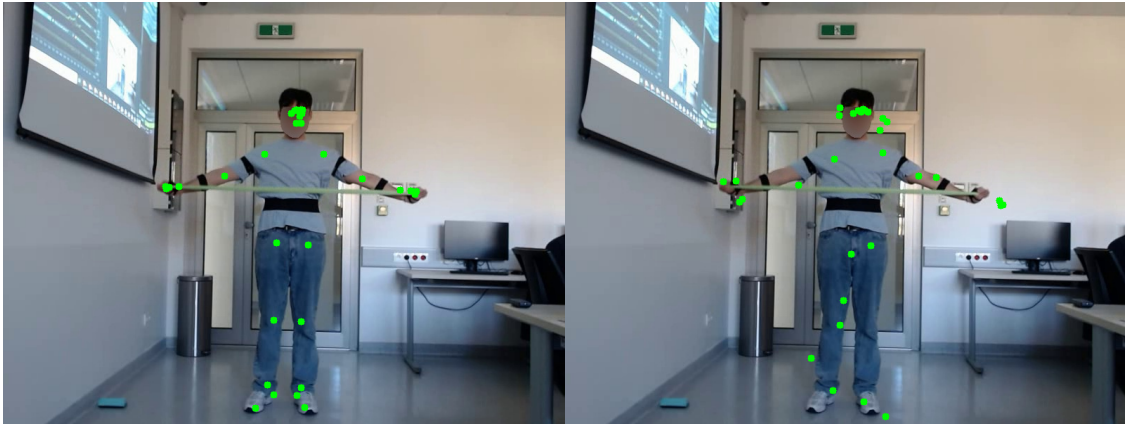
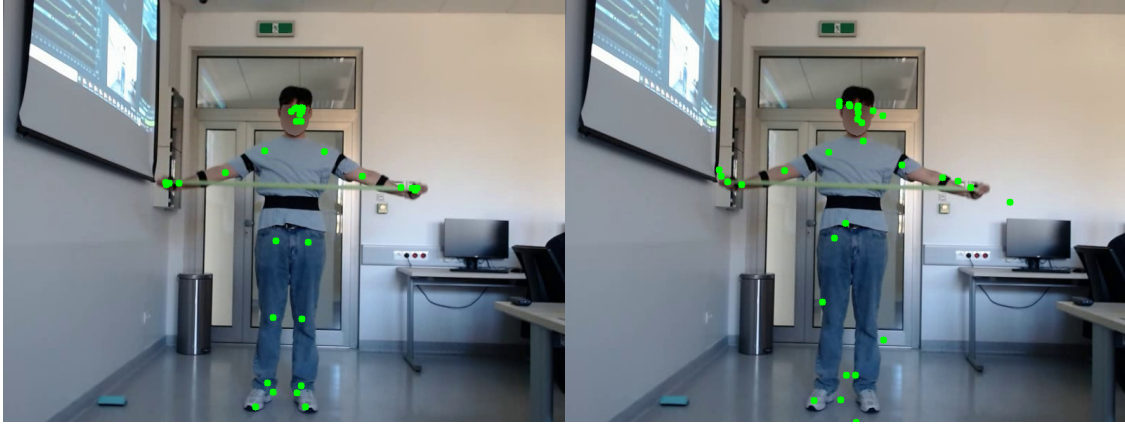
```

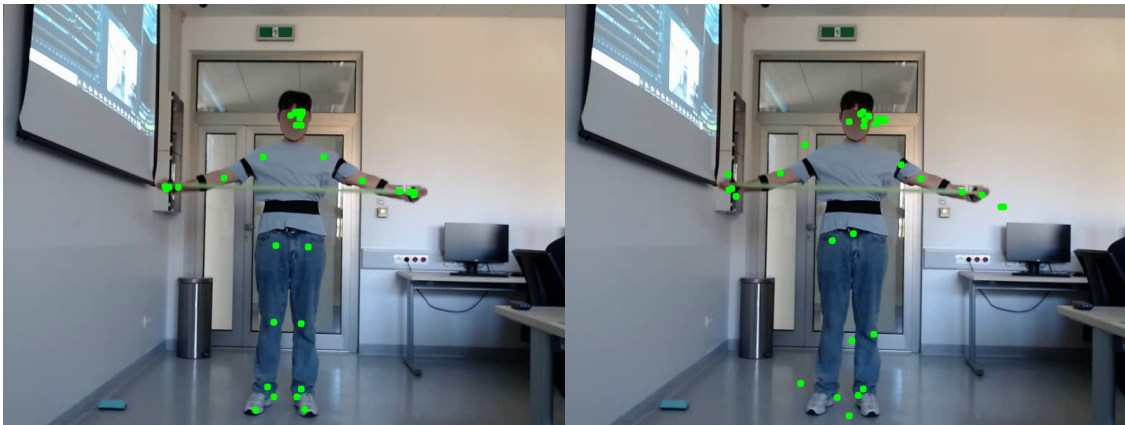
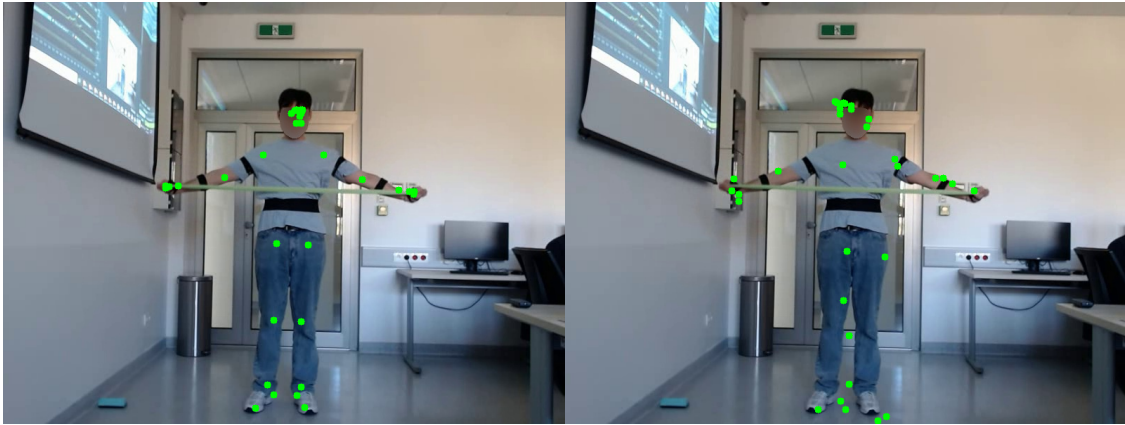
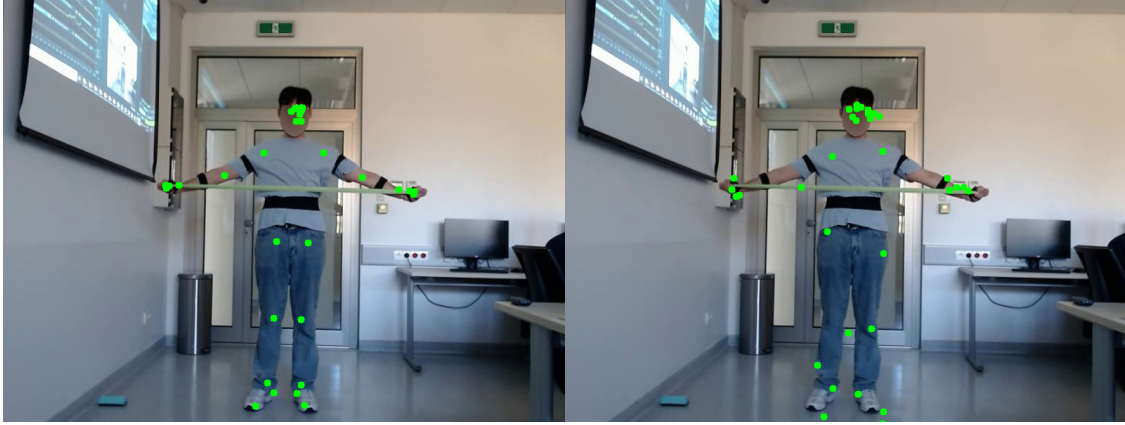
```
combined_image = cv2.hconcat([original_image_with_keypoints,   
↪ augmented_image_with_keypoints])  
  
cv2.imshow(combined_image)
```

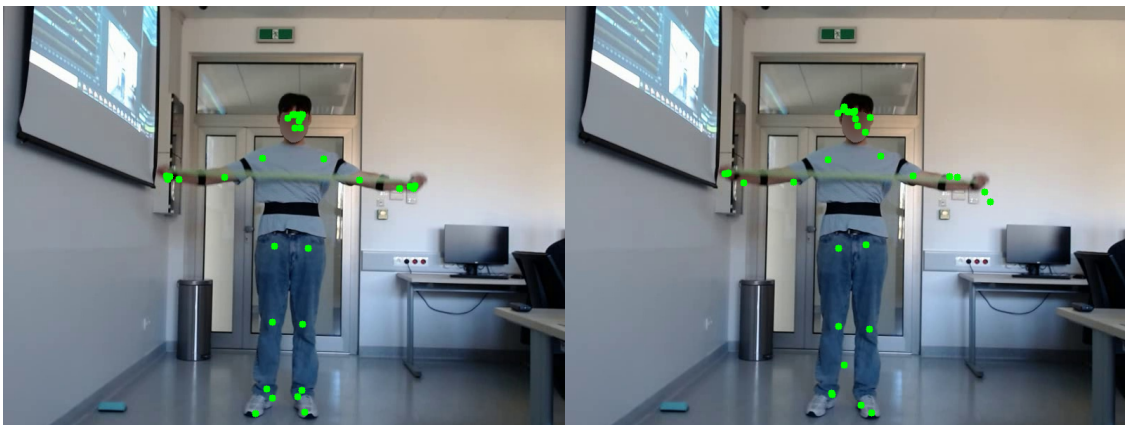
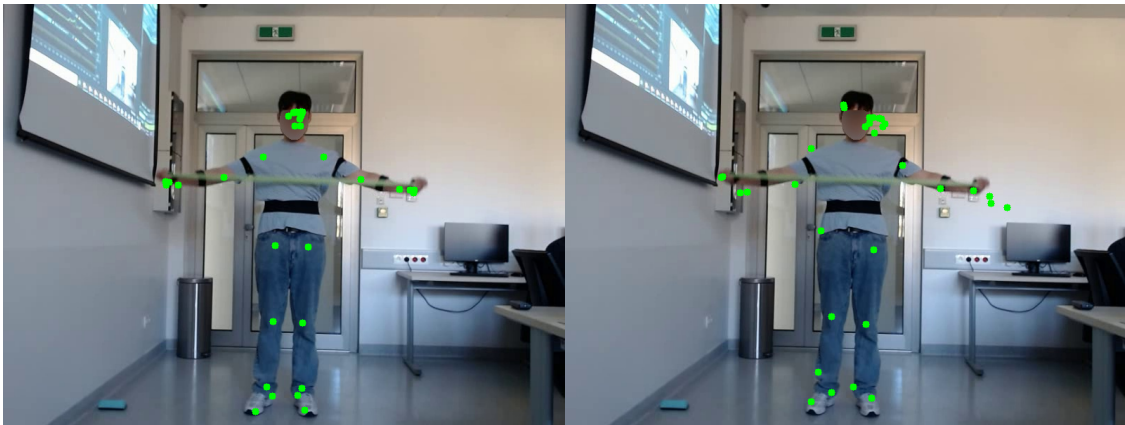
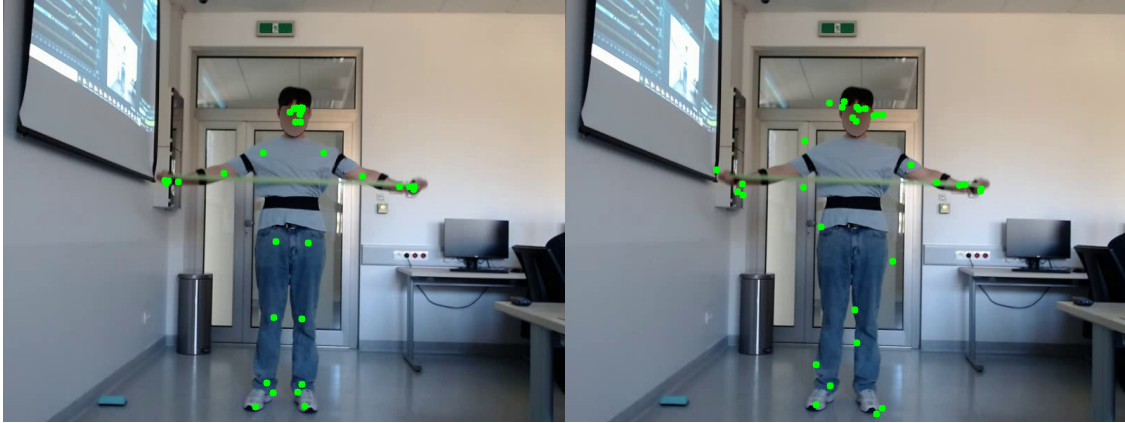


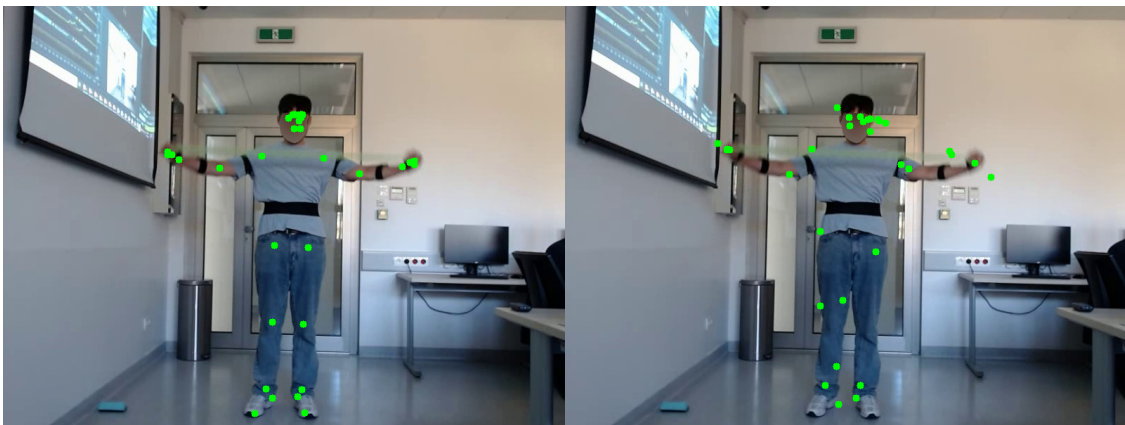
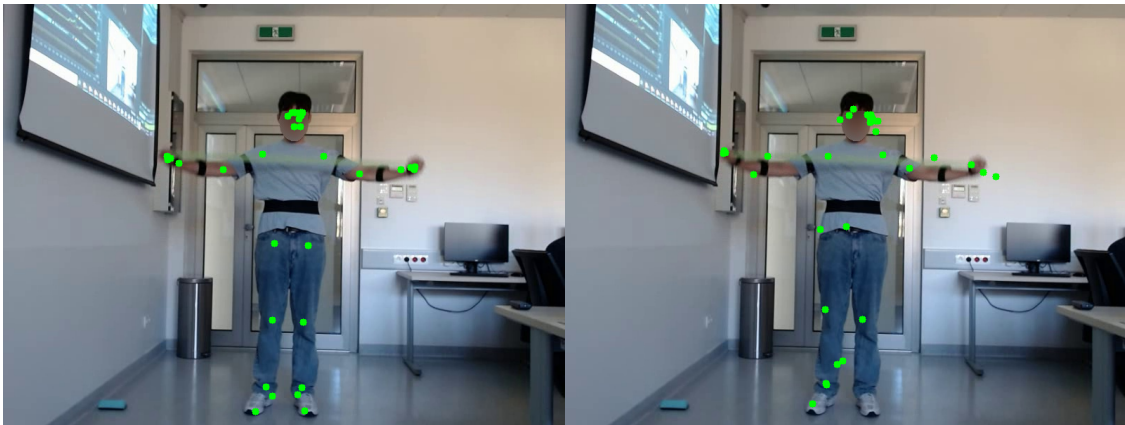
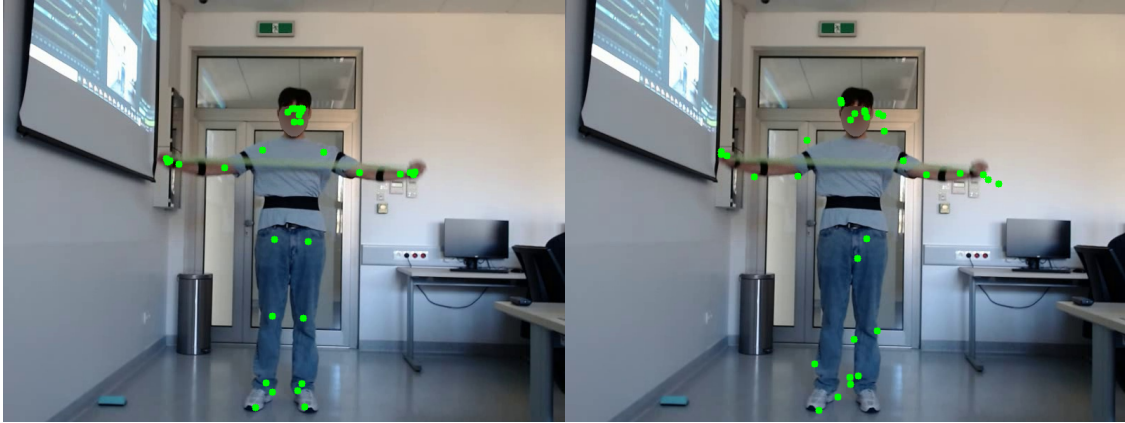


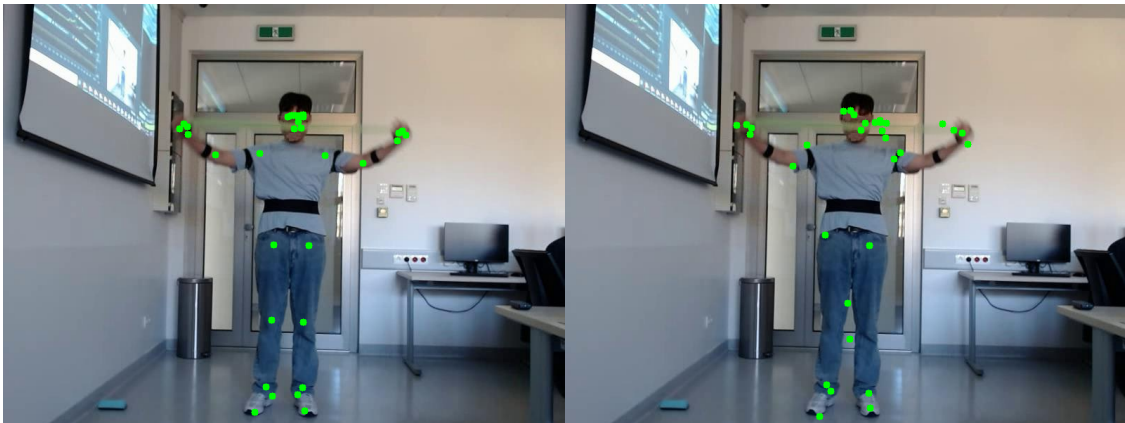
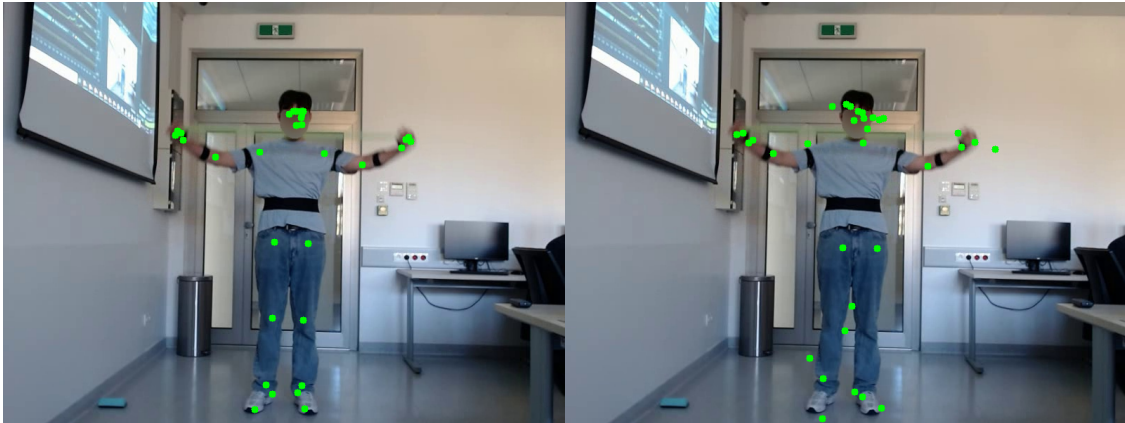
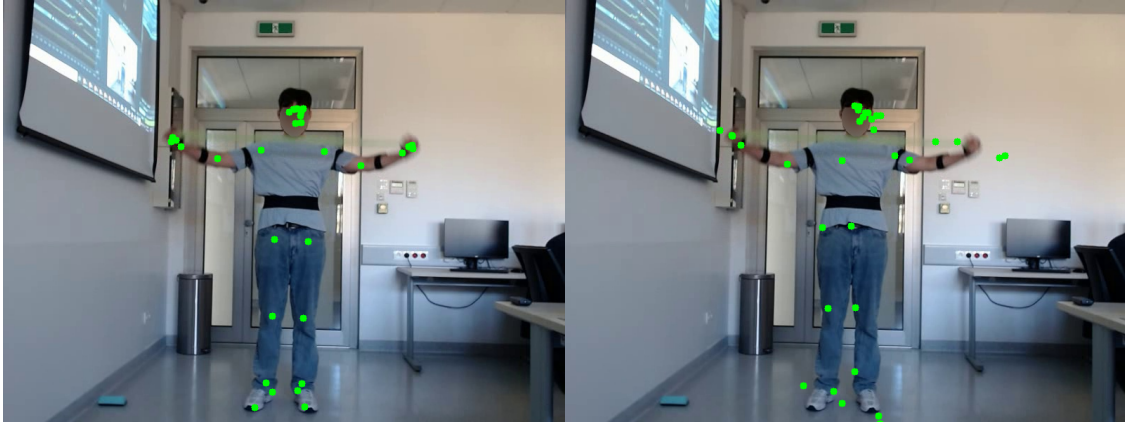


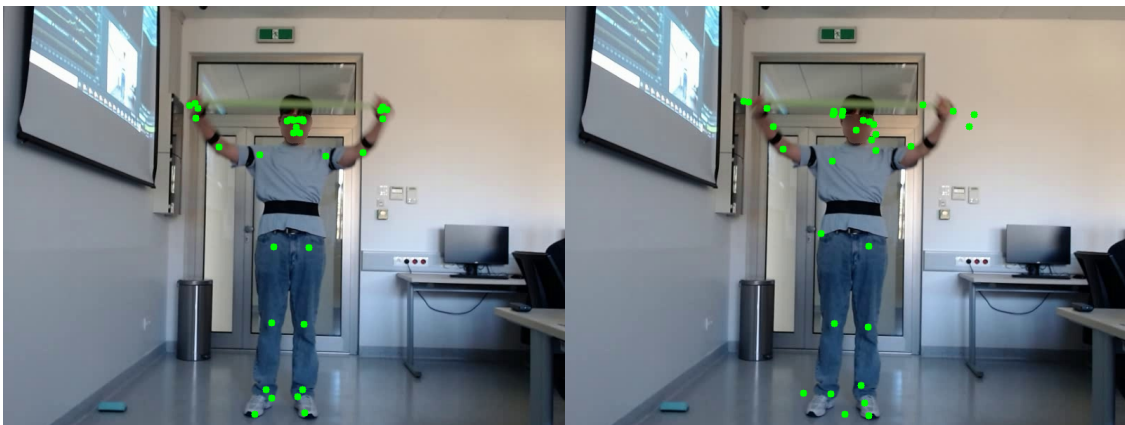
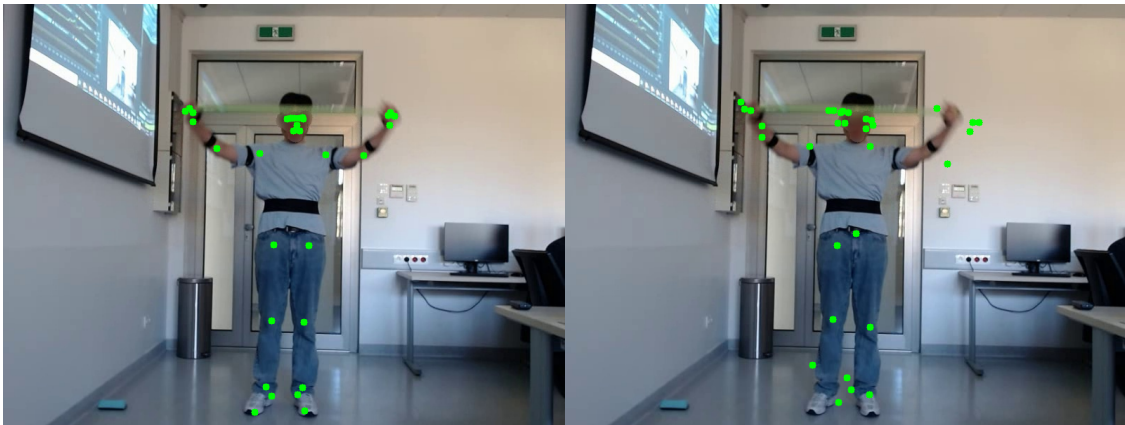
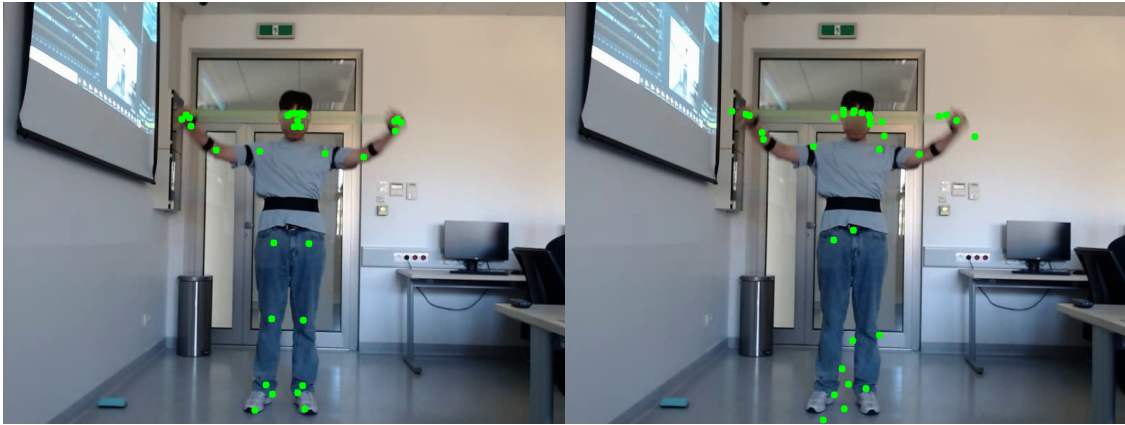


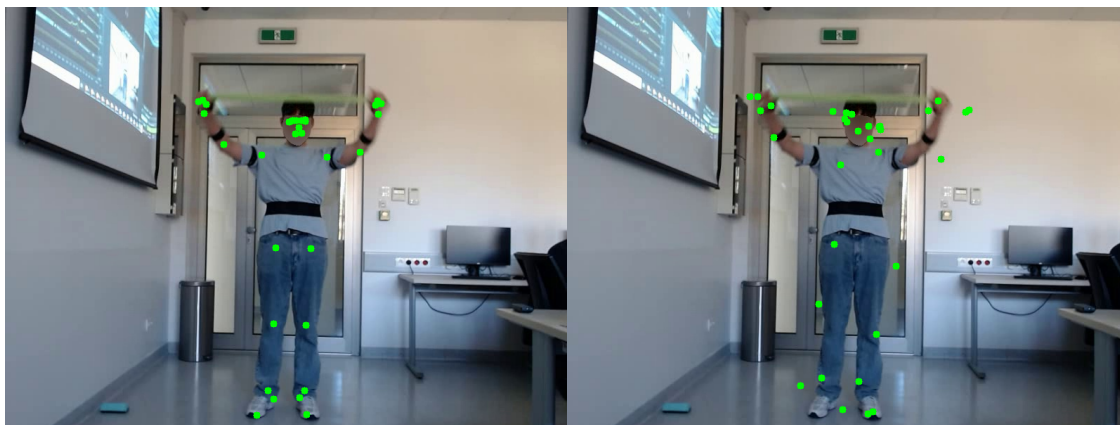












Wynikiem działania kodu jest wizualizacja szkieletu wejściowego oraz po augmentacji danych szkieletowych (w dwóch oknach). Oraz pliki json z keypointami przed oraz po augmentacji. W podanym przykładzie wykorzystano tylko kilkanaście losowych obrazków ponieważ dokument byłby zbyt duży aby wrzucić go na upel.

8glbibh0p

January 15, 2025

```
[7]: !pip install -q mediapipe
```

35.6/35.6 MB
28.3 MB/s eta 0:00:00

```
[8]: !wget -O pose_landmarker.task -q https://storage.googleapis.com/
↳mediapipe-models/pose_landmarker/pose_landmarker_heavy/float16/1/
↳pose_landmarker_heavy.task
```

```
[9]: !pip install torch-geometric
```

Collecting torch-geometric

Downloading torch_geometric-2.6.1-py3-none-any.whl.metadata (63 kB)

63.1/63.1 kB
1.5 MB/s eta 0:00:00

Requirement already satisfied: aiohttp in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (3.11.10)

Requirement already satisfied: fsspec in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (2024.10.0)

Requirement already satisfied: jinja2 in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (3.1.4)

Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (1.26.4)

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (5.9.5)

Requirement already satisfied: pyparsing in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (3.2.0)

Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (2.32.3)

Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (from torch-geometric) (4.67.1)

Requirement already satisfied: aiohappyeyeballs>=2.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (2.4.4)

Requirement already satisfied: aiosignal>=1.1.2 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.3.2)

Requirement already satisfied: async-timeout<6.0,>=4.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (4.0.3)

Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (24.3.0)

Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.5.0)

Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (6.1.0)

Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (0.2.1)

Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.10/dist-packages (from aiohttp->torch-geometric) (1.18.3)

Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from jinja2->torch-geometric) (3.0.2)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (3.4.0)

Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (3.10)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (2.2.3)

Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests->torch-geometric) (2024.12.14)

Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.10/dist-packages (from multidict<7.0,>=4.5->aiohttp->torch-geometric) (4.12.2)

Downloading torch_geometric-2.6.1-py3-none-any.whl (1.1 MB)

1.1/1.1 MB

12.2 MB/s eta 0:00:00

Installing collected packages: torch-geometric

Successfully installed torch-geometric-2.6.1

```
[10]: import pandas as pd
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
import cv2
import os
from google.colab.patches import cv2_imshow
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.data import Data
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx
import pandas as pd
```

```
import torch
import os
import random
```

```
[11]: import cv2
import os
import shutil
def read_frames_from_video():

    ranges = [420, 815, 1224, 1591, 1941, 2381, 2800, 3460, 3835, 4343, 4850,
↪5350, 5700, 6100, 6650, 6960, 7424, 7760, 8200]

    # List of ranges of frames to skip (e.g., skip frames from 50 to 100, and
↪200 to 250)
    skip_ranges = [(790, 815), (1211, 1233), (1920, 1990), (2348, 2428),
↪(2750,3000), (3370, 3490), (3805,3880), (4270, 4500), (4760, 4970), (5283,
↪5380), (5670, 5700), (5980, 6110), (6440, 6700), (6940, 7030), (7340, 7460),
↪(7730, 7785), (8120, 8250)]

    # Create the 'images' folder if it doesn't exist
    output_folder = "images"

    # Clear the 'images' folder if it exists, or create it
    if os.path.exists(output_folder):
        shutil.rmtree(output_folder) # Remove the folder and its contents
    os.makedirs(output_folder, exist_ok=True) # Create an empty folder

    vidcap = cv2.VideoCapture('d26arrd1_anon.mp4')
    success, image = vidcap.read()
    count = 0
    prefix_index = 0

    def is_skipped(frame_idx, skip_ranges):
        """Check if the frame index is within any of the skip ranges."""
        for start, end in skip_ranges:
            if start <= frame_idx <= end:
                return True
        return False

    while success:
        count += 1
        # Skip frames that fall within the skip_ranges
        if is_skipped(count, skip_ranges):
            success, image = vidcap.read()
            continue
```

```

    # Determine the current prefix based on the frame count
    while prefix_index < len(ranges) and count >= ranges[prefix_index]:
        prefix_index += 1

    # Create the filename with the appropriate prefix
    prefix = "{:02d}_".format(prefix_index) # Start from 00
    filename = os.path.join(output_folder, "{}{:04d}.jpg".format(prefix,
↪count))

    # Save the frame
    cv2.imwrite(filename, image)
    success, image = vidcap.read()

```

```
[12]: read_frames_from_video()
```

```
[13]: print(os.listdir("/content/images")[-20:-1])
```

```

['08_3622.jpg', '15_6903.jpg', '19_8520.jpg', '11_5101.jpg', '05_2252.jpg',
'15_6808.jpg', '08_3752.jpg', '13_5861.jpg', '01_0544.jpg', '17_7726.jpg',
'12_5483.jpg', '18_7992.jpg', '18_7936.jpg', '02_0859.jpg', '19_8321.jpg',
'13_5798.jpg', '10_4652.jpg', '01_0451.jpg', '17_7494.jpg']

```

```

[14]: from mediapipe import solutions
from mediapipe.framework.formats import landmark_pb2
import numpy as np

def draw_landmarks_on_image(rgb_image, detection_result):
    pose_landmarks_list = detection_result.pose_landmarks
    annotated_image = np.copy(rgb_image)

    # Loop through the detected poses to visualize.
    for idx in range(len(pose_landmarks_list)):
        pose_landmarks = pose_landmarks_list[idx]

        # Draw the pose landmarks.
        pose_landmarks_proto = landmark_pb2.NormalizedLandmarkList()
        pose_landmarks_proto.landmark.extend([
            landmark_pb2.NormalizedLandmark(x=landmark.x, y=landmark.y, z=landmark.z)
↪for landmark in pose_landmarks
        ])
        solutions.drawing_utils.draw_landmarks(
            annotated_image,
            pose_landmarks_proto,
            solutions.pose.POSE_CONNECTIONS,
            solutions.drawing_styles.get_default_pose_landmarks_style())
    return annotated_image

```



```

[16]: base_options = python.BaseOptions(model_asset_path='pose_landmarker.task')
options = vision.PoseLandmarkerOptions(
    base_options=base_options,
    output_segmentation_masks=True)
detector = vision.PoseLandmarker.create_from_options(options)
df_array = []
image_directory = "/content/images"
files = os.listdir(image_directory)
for i, file_name in enumerate(files):
    if file_name.endswith(('.jpg', '.png', '.jpeg')):
        image_path = os.path.join(image_directory, file_name)
        image = mp.Image.create_from_file(image_path)

        detection_result = detector.detect(image)

        indices_to_keep = [24, 23, 12, 11, 13, 15, 14, 16]
        if i == 0:
            annotated_image = draw_landmarks_on_image(image.numpy_view(),
↳detection_result)
            annotated_image = cv2.cvtColor(annotated_image, cv2.COLOR_RGB2BGR)

            # Display the annotated image (or save it if needed).
            cv2.imshow(annotated_image)
            cv2.waitKey(0)
            cv2.destroyAllWindows()
            # Filter landmarks
            # print(detection_result.pose_landmarks[0])
            if len(detection_result.pose_landmarks) == 0: continue
            if len(detection_result.pose_landmarks[0]) != 33: continue
            detection_result.pose_landmarks[0] = [detection_result.
↳pose_landmarks[0][i] for i in indices_to_keep]

            # print(detection_result.pose_landmarks[0])

            # STEP 5: Process the detection result. In this case, visualize it.

            # Display the annotated image (or save it if needed).

            data = {
                # "Label": int(file_name[:2]),
                "Index": indices_to_keep,
                "X": [landmark.x for landmark in detection_result.
↳pose_landmarks[0]],
                "Y": [landmark.y for landmark in detection_result.pose_landmarks[0]]
            }
            df_array.append((pd.DataFrame(data), int(file_name[:2])))

```

```
[ ]: def cache_each_dataframe_to_csv(df_array, output_dir):
    """
    Saves each DataFrame in the df_array to a separate CSV file.

    :param df_array: List of tuples, where each tuple contains a DataFrame and
    ↪ an integer label.
    :param output_dir: Directory where the CSV files will be saved.
    """
    # Ensure the output directory exists
    os.makedirs(output_dir, exist_ok=True)

    for i, (df, label) in enumerate(df_array):
        # Create a unique filename for each DataFrame
        file_name = f"data_{label}_{i}.csv"
        file_path = os.path.join(output_dir, file_name)

        # Save the DataFrame to the CSV file
        df.to_csv(file_path, index=False)

        print(f"Saved DataFrame {i} with label {label} to {file_path}.")
    cache_each_dataframe_to_csv(df_array, "/content/coordinates")
```

```
[18]: import zipfile
def zip_folder(folder_path, zip_file_path):
    """
    Compresses a folder into a ZIP file.

    :param folder_path: Path to the folder to be compressed.
    :param zip_file_path: Path to the output ZIP file.
    """
    # Ensure the folder exists
    if not os.path.exists(folder_path):
        print(f"Folder '{folder_path}' does not exist.")
        return

    # Create a ZIP file
    with zipfile.ZipFile(zip_file_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
        # Walk through all files and subfolders in the folder
        for root, dirs, files in os.walk(folder_path):
            for file in files:
                # Full path to the file
                file_path = os.path.join(root, file)
                # Relative path for the ZIP archive
                arcname = os.path.relpath(file_path, folder_path)
                # Add file to the ZIP archive
                zipf.write(file_path, arcname)
```

```

    print(f"Folder '{folder_path}' successfully compressed into_
    ↪ '{zip_file_path}'.")
zip_folder("/content/coordinates", "/content/coordinates.zip")

```

Folder '/content/coordinates' successfully compressed into
'/content/coordinates.zip'.

```

[19]: # Step 1: Generate Toy Data for Human Skeleton Motion
num_frames = 100 # Number of frames in the sequence
num_joints = 8 # Number of joints in the skeleton
num_features = 2 # (x, y) positions for each joint
num_classes = 20 # 3 classes for human actions (e.g., walk, run, jump)
# edge_index = torch.tensor([[16, 14, 12, 12, 24, 23, 11, 13],
#                             [14, 12, 11, 24, 23, 11, 13, 15]], dtype=torch.
    ↪ long)
edge_index = torch.tensor([[7, 6, 5, 5, 1, 0, 2, 3],
                           [6, 5, 2, 1, 0, 2, 3, 4]], dtype=torch.long)
# edge_index = torch.tensor([[0, 1, 2, 3, 4, 5, 6, 7],
#                             [1, 2, 3, 4, 5, 6, 7, 0]], dtype=torch.long)

```

```

[20]: def convert_to_tensors(data_frames):
    data_objects = [] # Initialize an empty list to store Data objects
    for df, label in data_frames: # Unpack the DataFrame and label from each_
    ↪ element
        x_data = []

        for i in range(len(df["X"])):
            x_data.append((df["X"][i], df["Y"][i]))

        x_data = torch.tensor(x_data, dtype=torch.float32)
        y_data = torch.tensor([label], dtype=torch.long)

        # Create the PyTorch Geometric Data object
        data_object = Data(x=x_data, edge_index=edge_index, y=y_data)
        data_objects.append(data_object)
    return data_objects

```

```

[ ]: # train_data_frames = convert_to_tensors(df_array)

```

```

[21]: class STGCN(nn.Module):
    def __init__(self, in_channels, out_channels, num_joints, num_frames):
        super(STGCN, self).__init__()
        self.conv1 = GCNConv(in_channels, 64) # First graph convolution layer
        self.conv2 = GCNConv(64, 128) # Second graph convolution layer
        self.temporal_conv = nn.Conv2d(128, 128, kernel_size=(3, 1),_
    ↪ padding=(1, 0)) # Temporal convolution

```

```

        self.fc = nn.Linear(128 * num_frames * num_joints, out_channels) #
    ↪ Fully connected layer for classification

    def forward(self, data):
        x, edge_index = data.x, data.edge_index
        # Graph convolution layers
        x = F.relu(self.conv1(x, edge_index))
        x = F.relu(self.conv2(x, edge_index))

        # Reshape x to include the time dimension (num_frames)
        # We will repeat the graph features across time frames
        x = x.unsqueeze(0) # Add batch dimension: shape becomes [1,
    ↪ num_joints, 128]
        x = x.permute(0, 2, 1) # Change shape to [1, 128, num_joints]
    ↪ (channels, joints)
        x = x.repeat(1, 1, num_frames) # Repeat across time frames: [1, 128,
    ↪ num_joints, num_frames]

        # Apply the temporal convolution (across time)
        x = x.unsqueeze(3) # Add time_steps dimension: [1, 128, num_joints,
    ↪ num_frames, 1]
        x = F.relu(self.temporal_conv(x))

        # Flatten the output for the fully connected layer
        x = x.view(x.size(0), -1) # Flatten all but the batch dimension

        # Output classification
        x = self.fc(x)
        return x

```

```

[22]: # Step 4: Training the ST-GCN

# Initialize the model, optimizer, and loss function
model = STGCN(in_channels=num_features, out_channels=num_classes,
    ↪ num_joints=num_joints, num_frames=num_frames)
optimizer = optim.Adam(model.parameters(), lr=0.001)
criterion = nn.CrossEntropyLoss()

```

```

[23]: # Function to train the model with a sequence of frames
def train(model, data_frames, optimizer, criterion, epochs=100):
    model.train()
    for epoch in range(epochs):
        optimizer.zero_grad()
        total_loss = 0

        # Loop through the sequence of frames

```

```

for frame in data_frames:
    # Forward pass for each frame in the sequence
    out = model(frame)

    # Compute loss for the current frame
    loss = criterion(out, frame.y)
    total_loss += loss

    # Backpropagation on the aggregated loss
    total_loss.backward()
    optimizer.step()

if epoch % 20 == 0:
    print(f'Epoch {epoch+1}, Total Loss: {total_loss.item()}')

# Step 5: Evaluation

def test(model, data_frame):
    # model.eval()
    # with torch.no_grad():
    #     out = model(data_frame) # Run the forward pass
    #     pred = out.argmax(dim=1) # Get the predicted class
    #     correct = (pred == data_frame.y).sum() # Compare with true label
    #     accuracy = correct / len(data_frame.y)
    #     return accuracy.item()
def test(model, test_data_frames):
    model.eval()
    total_correct = 0
    total_samples = 0

    with torch.no_grad():
        for data_frame in test_data_frames:
            # Run the forward pass for each data frame
            out = model(data_frame)
            pred = out.argmax(dim=1) # Get the predicted class
            correct = (pred == data_frame.y).sum() # Compare with true label
            total_correct += correct.item()
            total_samples += len(data_frame.y)

    accuracy = total_correct / total_samples
    return accuracy

```

```

[24]: # Ustawienie proporcji podziału
test_ratio = 0.2

```

```

# Obliczenie liczby elementów w zbiorze testowym
total_data = len(df_array)
test_size = int(total_data * test_ratio)

# Podział danych
test_data_objects = df_array[:test_size] # Pierwsze 20% danych jako zbiór
↳ testowy
train_data_objects = df_array[test_size:] # Pozostałe 80% danych jako zbiór
↳ treningowy

# Konwersja ramek danych na obiekty Data
train_data_frames = convert_to_tensors(train_data_objects)
test_data_frames = convert_to_tensors(test_data_objects)

```

```

[26]: train(model, train_data_frames, optimizer, criterion, epochs=100)

```

```

Test the model on the first frame (toy data)
accuracy = test(model, test_data_frames)
print(f'Accuracy on test data: {accuracy * 100:.2f}%',)

```

```

Epoch 1, Total Loss: 541.756000
Epoch 2, Total Loss: 492.997960
Epoch 3, Total Loss: 448.628144
Epoch 4, Total Loss: 408.251611
Epoch 5, Total Loss: 371.508966
Epoch 6, Total Loss: 338.073159
Epoch 7, Total Loss: 307.646575
Epoch 8, Total Loss: 279.958383
Epoch 9, Total Loss: 254.762128
Epoch 10, Total Loss: 231.833537
Epoch 11, Total Loss: 210.968518
Epoch 12, Total Loss: 191.981352
Epoch 13, Total Loss: 174.703030
Epoch 14, Total Loss: 158.979757
Epoch 15, Total Loss: 144.671579
Epoch 16, Total Loss: 131.651137
Epoch 17, Total Loss: 119.802535
Epoch 18, Total Loss: 109.020307
Epoch 19, Total Loss: 99.208479
Epoch 20, Total Loss: 90.279716
Epoch 21, Total Loss: 82.154542
Epoch 22, Total Loss: 74.760633
Epoch 23, Total Loss: 68.032176
Epoch 24, Total Loss: 61.909280
Epoch 25, Total Loss: 56.337445
Epoch 26, Total Loss: 51.267075
Epoch 27, Total Loss: 46.653038

```

Epoch 28, Total Loss: 42.454265
Epoch 29, Total Loss: 38.633381
Epoch 30, Total Loss: 35.156377
Accuracy on test data: 77.41%